

EECS 836: Machine Learning

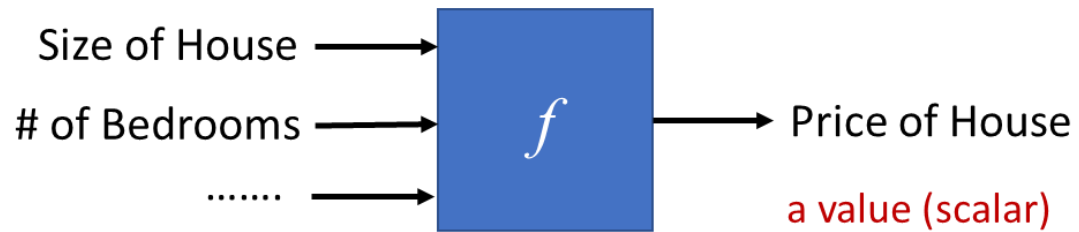
Zijun Yao

Assistant Professor, EECS Department
The University of Kansas

Agenda

- Neural Networks Overview
- Artificial Neuron
- Multi-Layer Perceptron (MLP) Networks
- Backpropagation
- Optimization

Represent linear regression in graph



Size of House	Lot Size (acre)	# of Bedrooms	# of Bathrooms	Price of House
950	2.5	2	1	\$127,325
1,535	1.5	2	2	\$156,570
1,605	2.25	3	1.5	\$158,895
1,905	2.5	2	1.5	\$200,025
2,057	2.25	3	2	\$230,384
2,227	2.75	3	2	\$233,835
3,150	1	4	2	\$261,420
3,620	3	4	3	\$433,500

Feature x

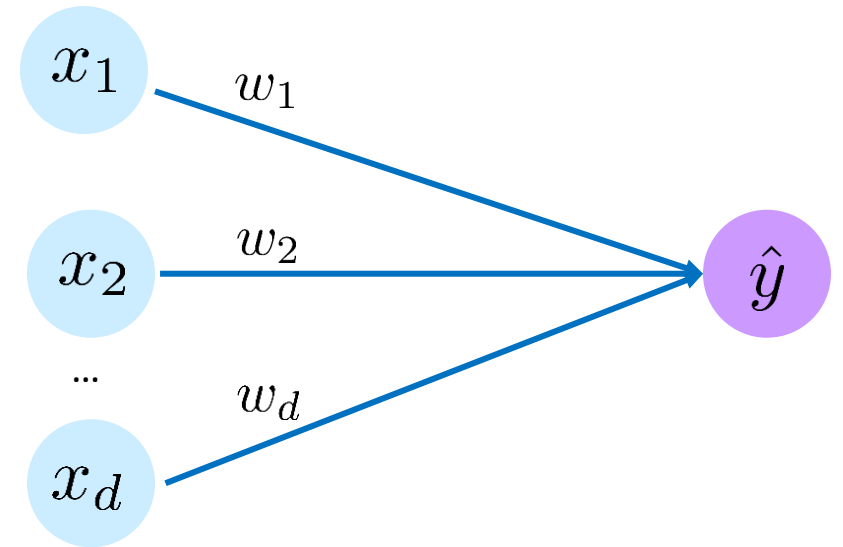
Target y

Output
↓
 $\hat{y} = g\left(\sum_{i=1}^d x_i \cdot w_i\right)$

Inputs
↓
 x_i

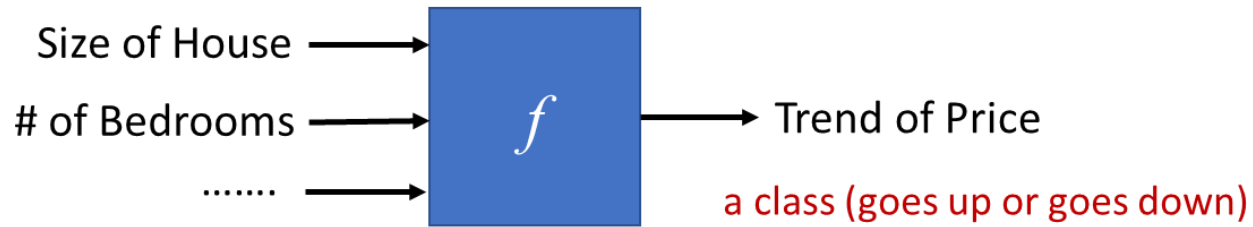
Weights
↓
 w_i

where $g(z) = z$



Let's don't consider bias at this point, we'll add it later!

Then let's see logistic regression



Size of House	Lot Size (acre)	# of Bedrooms	# of Bathrooms	Trend of Price
950	2.5	2	1	Down
1,535	1.5	2	2	Up
1,605	2.25	3	1.5	Down
1,905	2.5	2	1.5	Down
2,057	2.25	3	2	Up
2,227	2.75	3	2	Up
3,150	1	4	2	Down
3,620	3	4	3	Up

Feature x

Target y

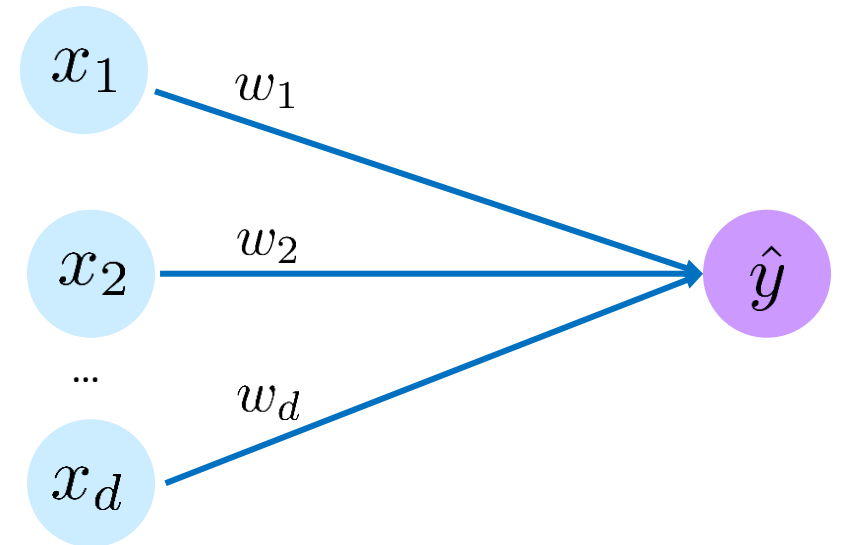
Output

Inputs

Weights

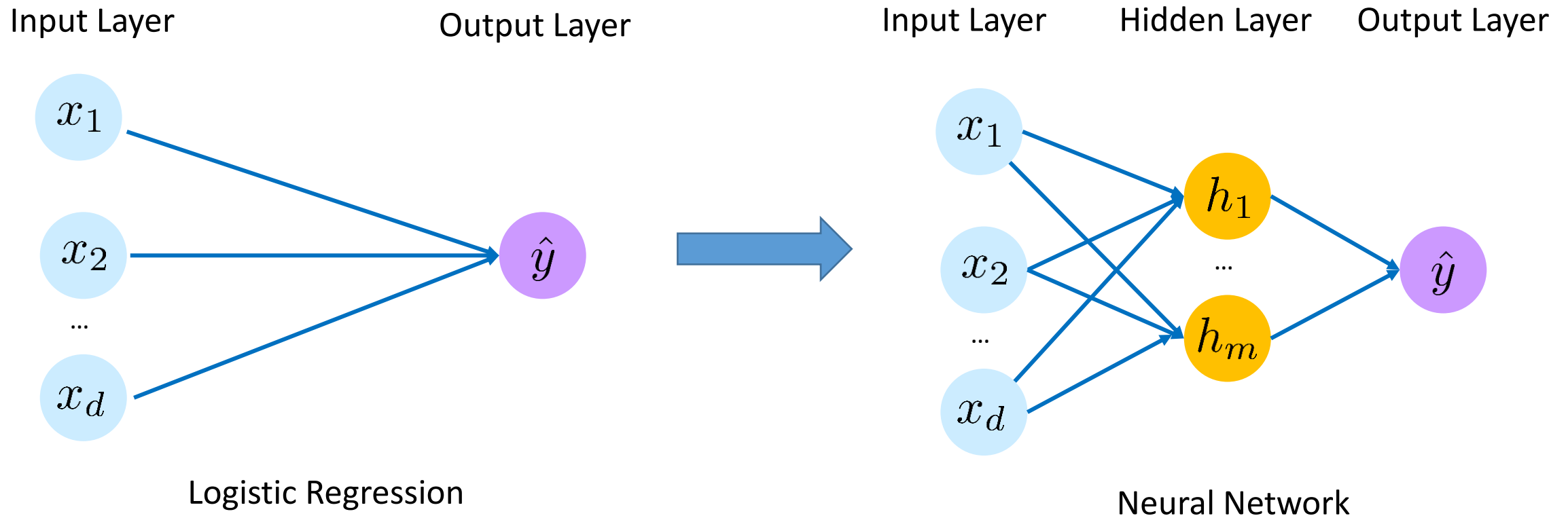
$$\hat{y} = g\left(\sum_{i=1}^d x_i \cdot w_i\right)$$

where $g(z) = \frac{1}{1 + \exp^{-z}}$



Neural Network

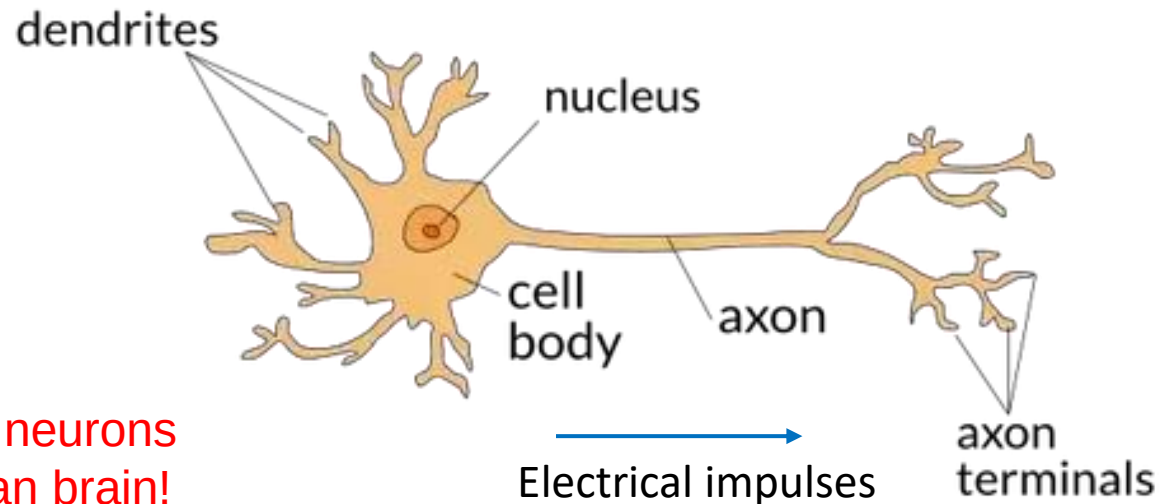
- Hidden Layers – a “container” for holding a collection of neurons, where each neuron performs an individual function (e.g., logistic regression)



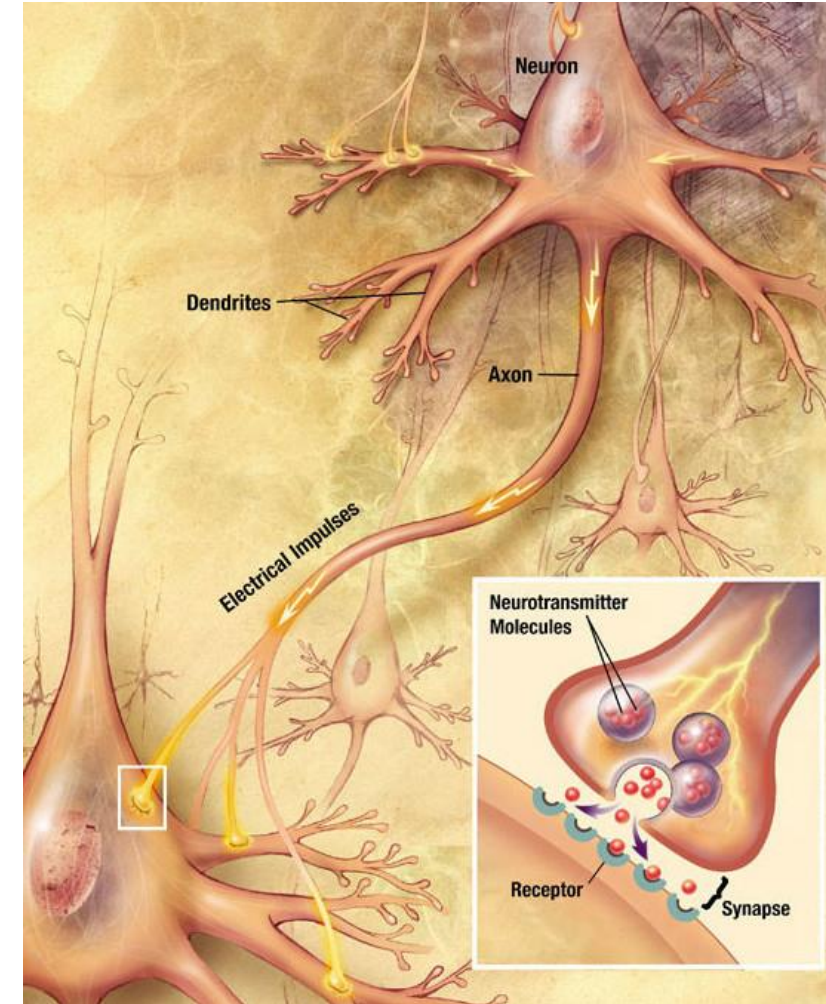
From Biological to Artificial

Brain function occurs as result of neuron firing

- **Dendrites** are used to **connected** with other neurons
- **Cell body** **controls** the cell's activities by accumulate impulses received from dendrites
- **Axon** is an extension from the cell body that **passes** impulse to other neurons.



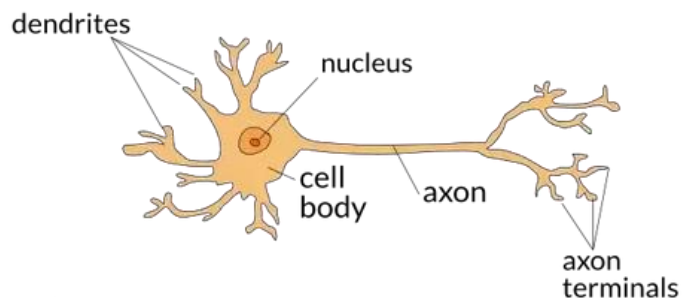
86 billion neurons
in a human brain!



From Biological to Artificial

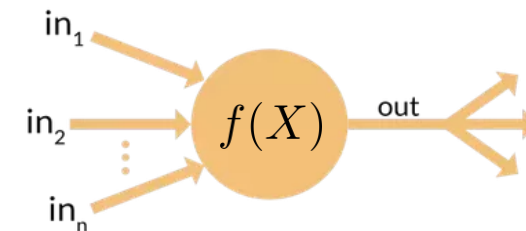
Biological “Model”

- **Neuron:** an excitable cell
- **Dendrites:** received impulse from other neurons
- A neuron fires an **electrical impulse** when a sufficient voltage change occurs
- **Biological Neural Network:** collection of neurons along some pathway through the brain



Artificial Model

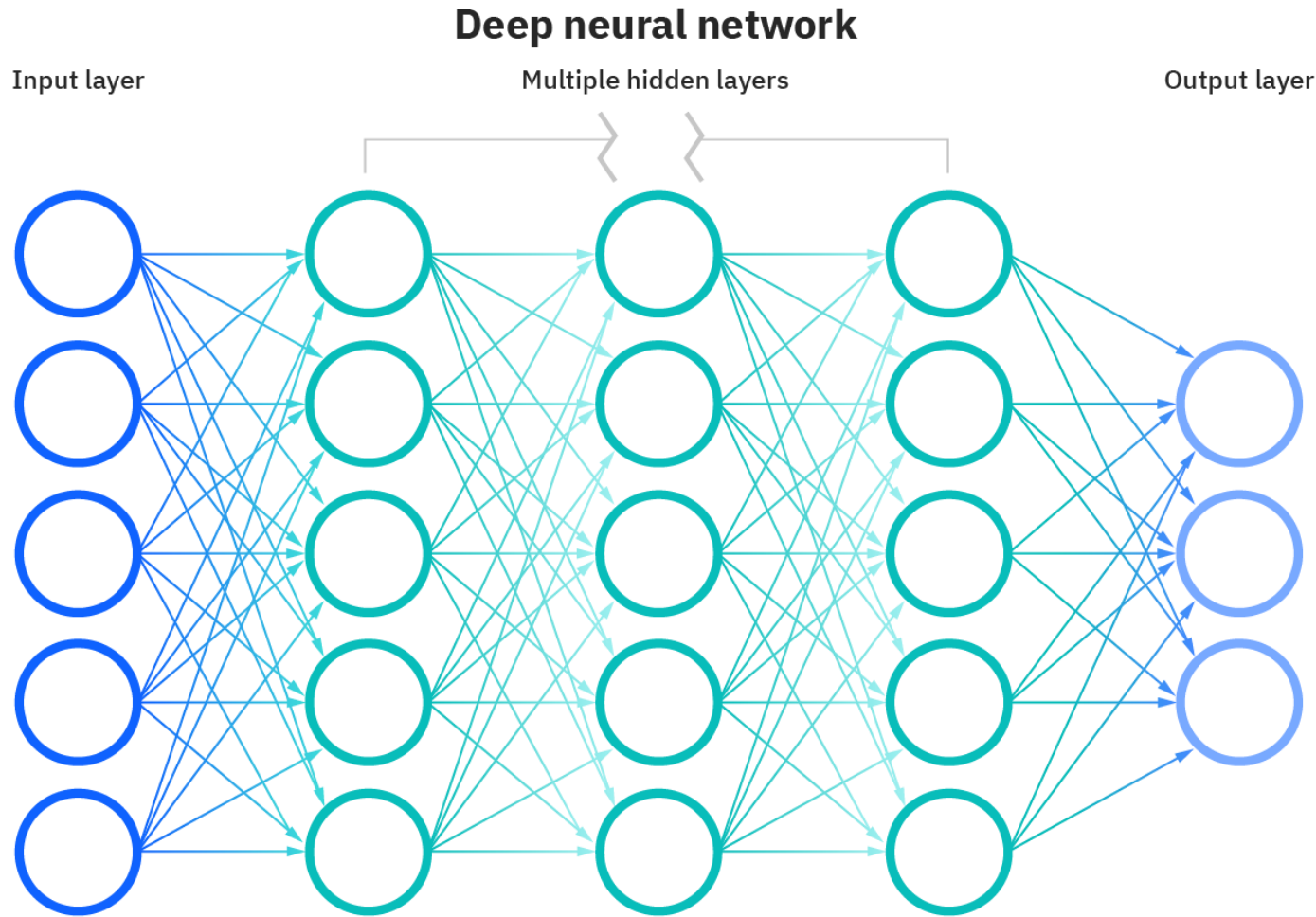
- **Neuron:** a node in a directed graph
- **Weight:** multiplier on each edge with inputs
- **Activation function:** nonlinear threshold function, allowing a neuron to “fire” when input value is sufficient high
- **Artificial Neural Network:** collection of neurons in a DAG, which define a differentiable function



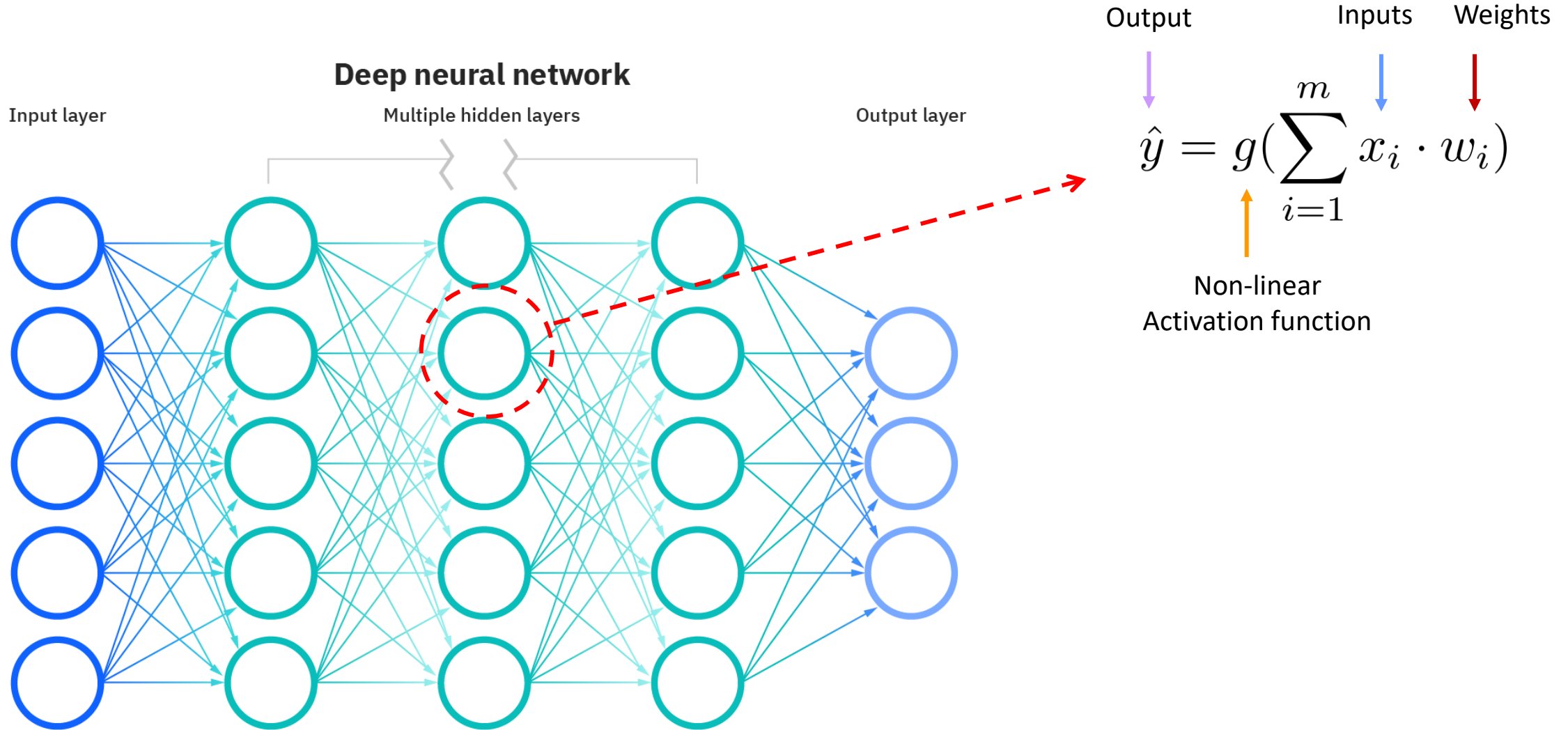
Agenda

- Neural Networks Overview
- **Artificial Neuron**
- Multi-Layer Perceptron (MLP) Networks
- Backpropagation
- Optimization

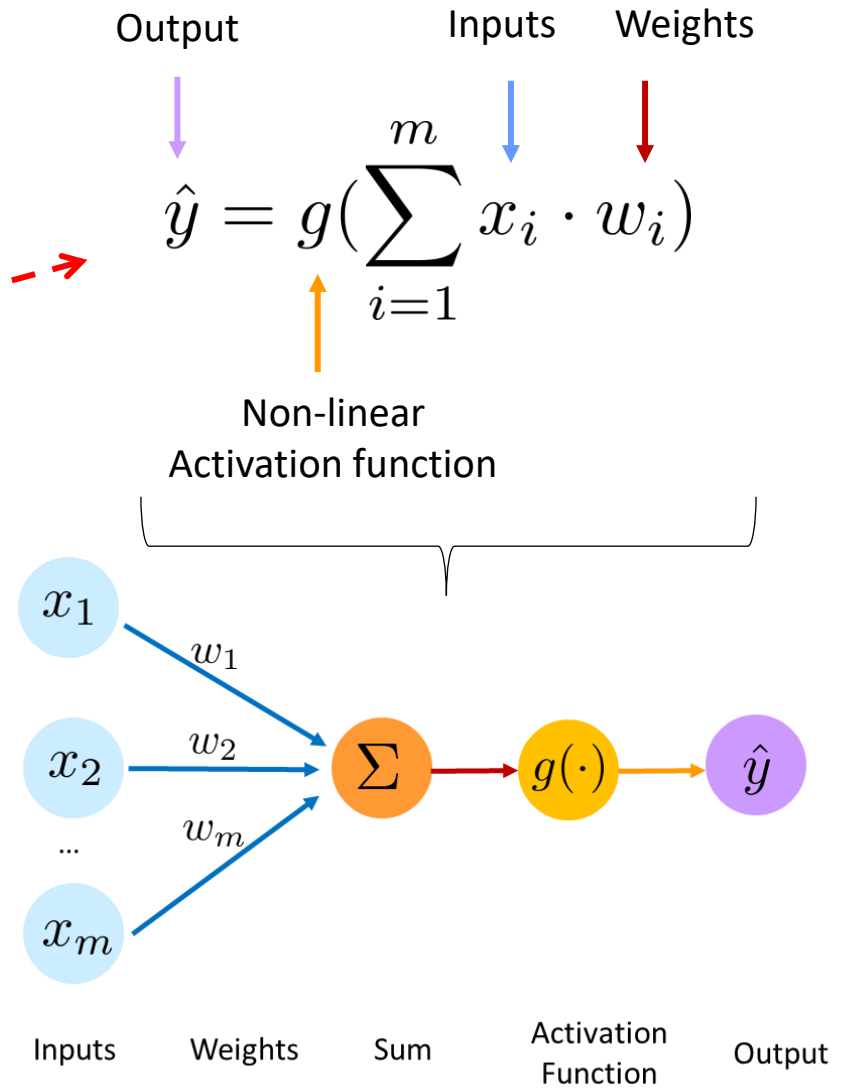
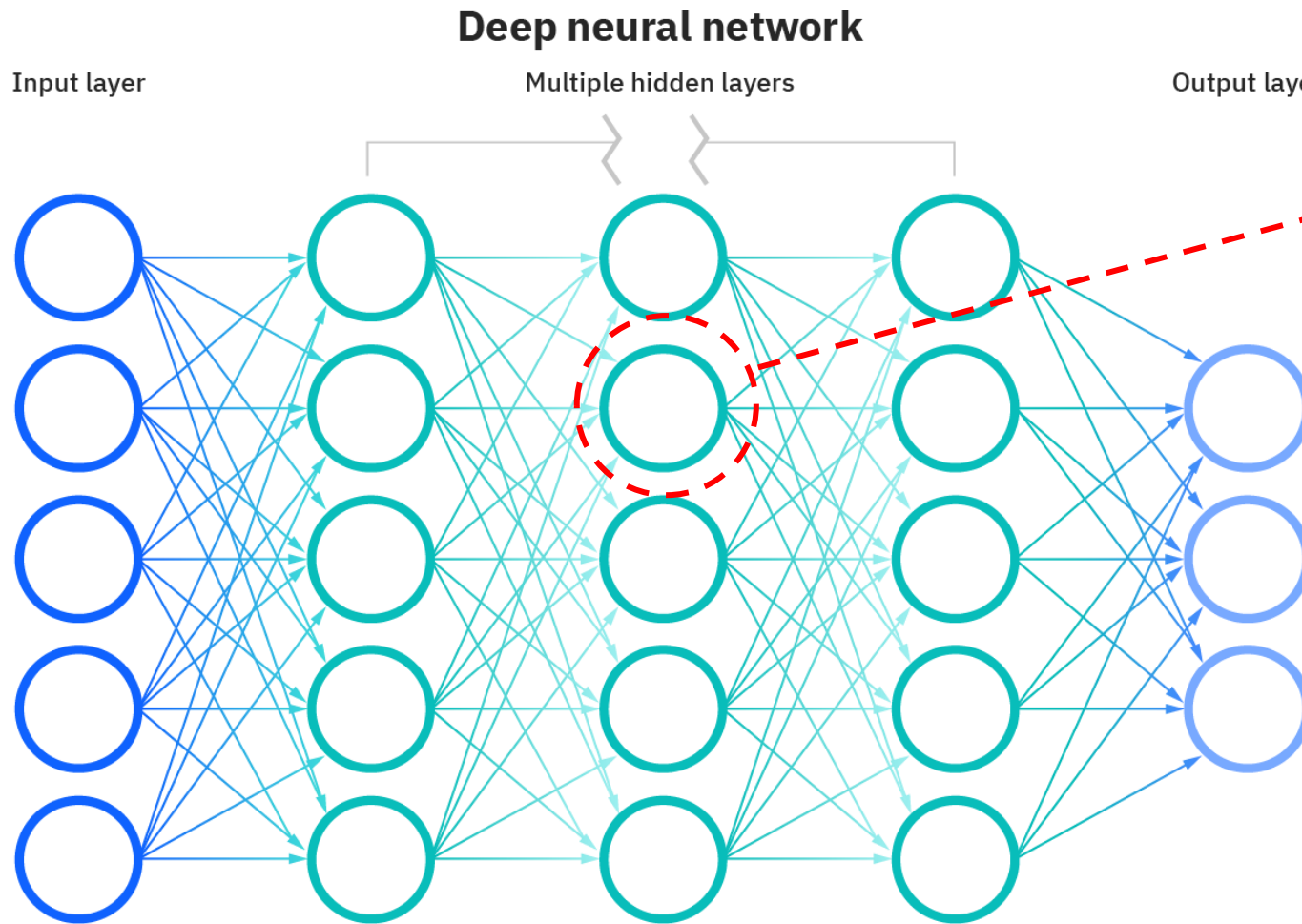
Zoom-in a single neuron



Zoom-in a single neuron



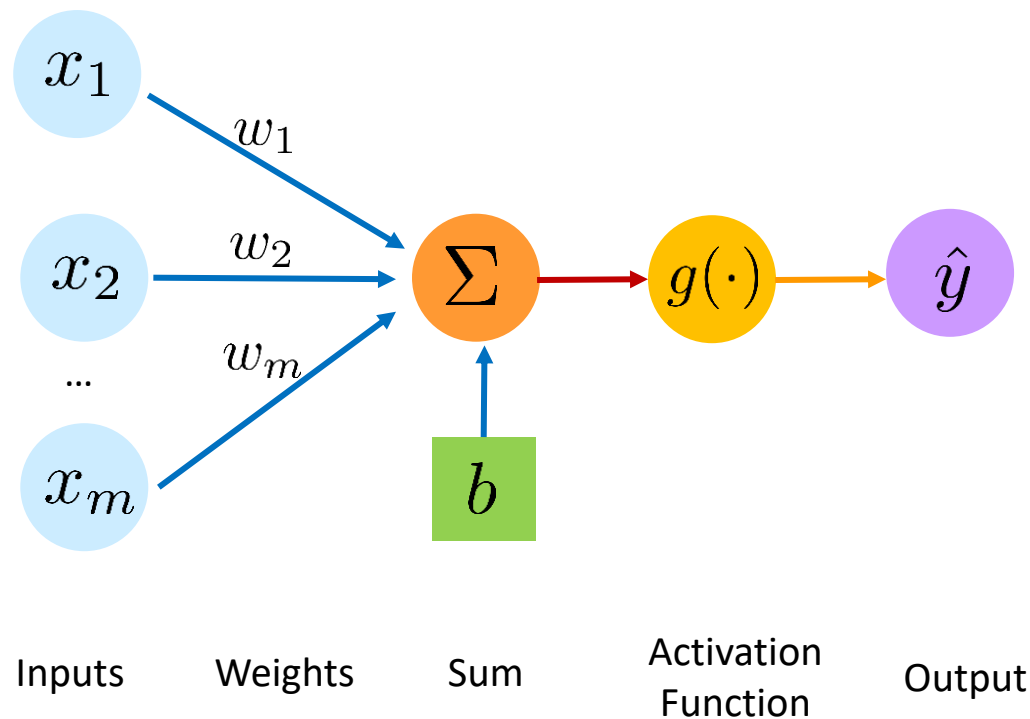
Zoom-in a single neuron



Detail steps within a single neuron

Feed-forward in neuron

- Input is fed in the forward direction through the network
- A neuron accepts the input data, processes it, and passes to the successive neuron

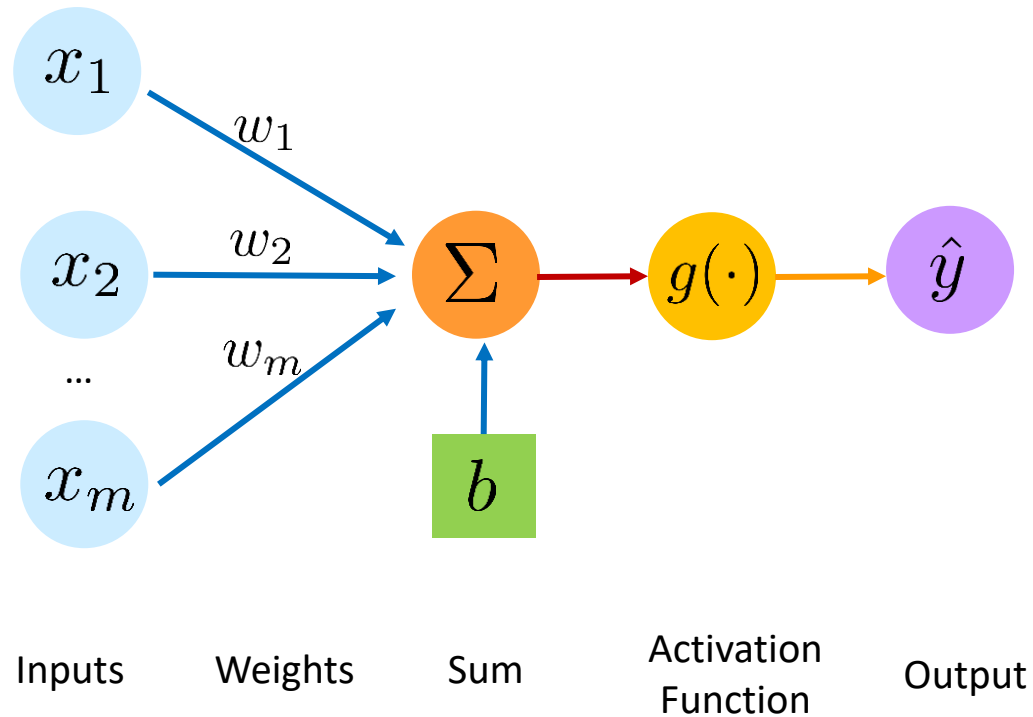


Output Bias Inputs Weights

$$\hat{y} = g\left(b + \sum_{i=1}^m x_i \cdot w_i\right)$$

Non-linear
Activation function

Feed-forward in neuron

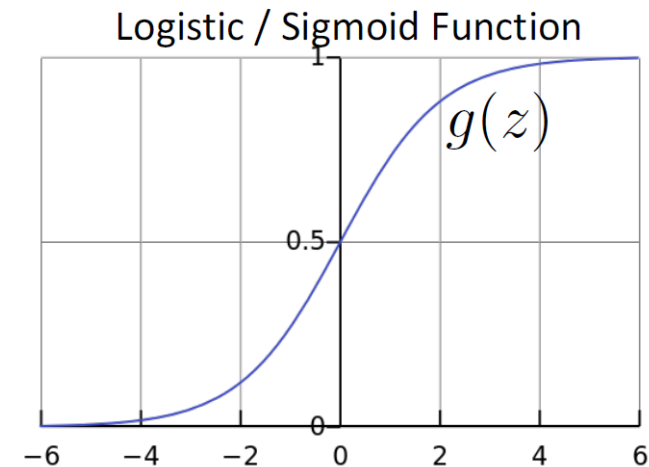


Activation Functions

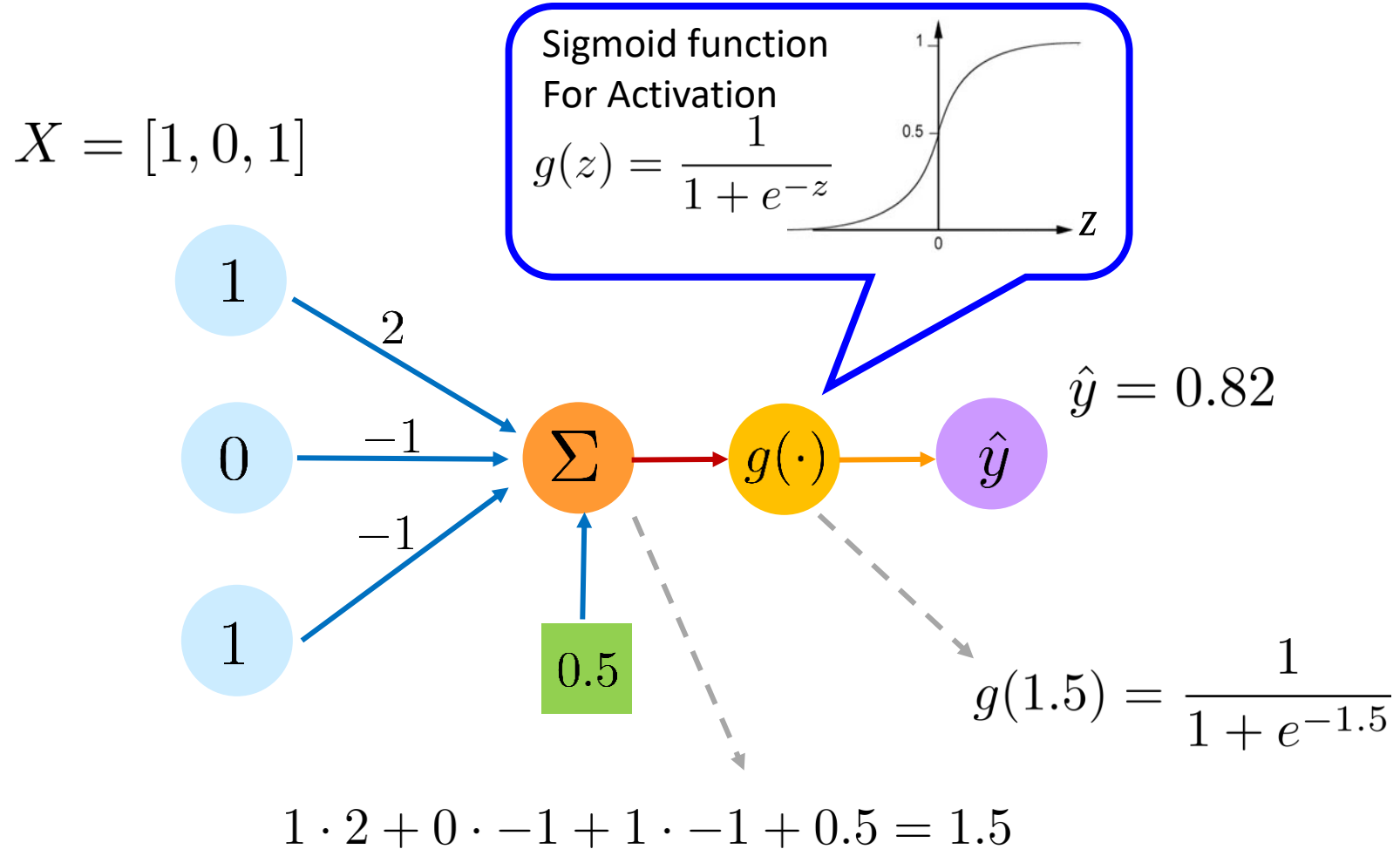
$$\hat{y} = g\left(b + \sum_{i=1}^m x_i \cdot w_i\right)$$

Example: Sigmoid Function

$$g(z) = \frac{1}{1 + e^{-z}}$$

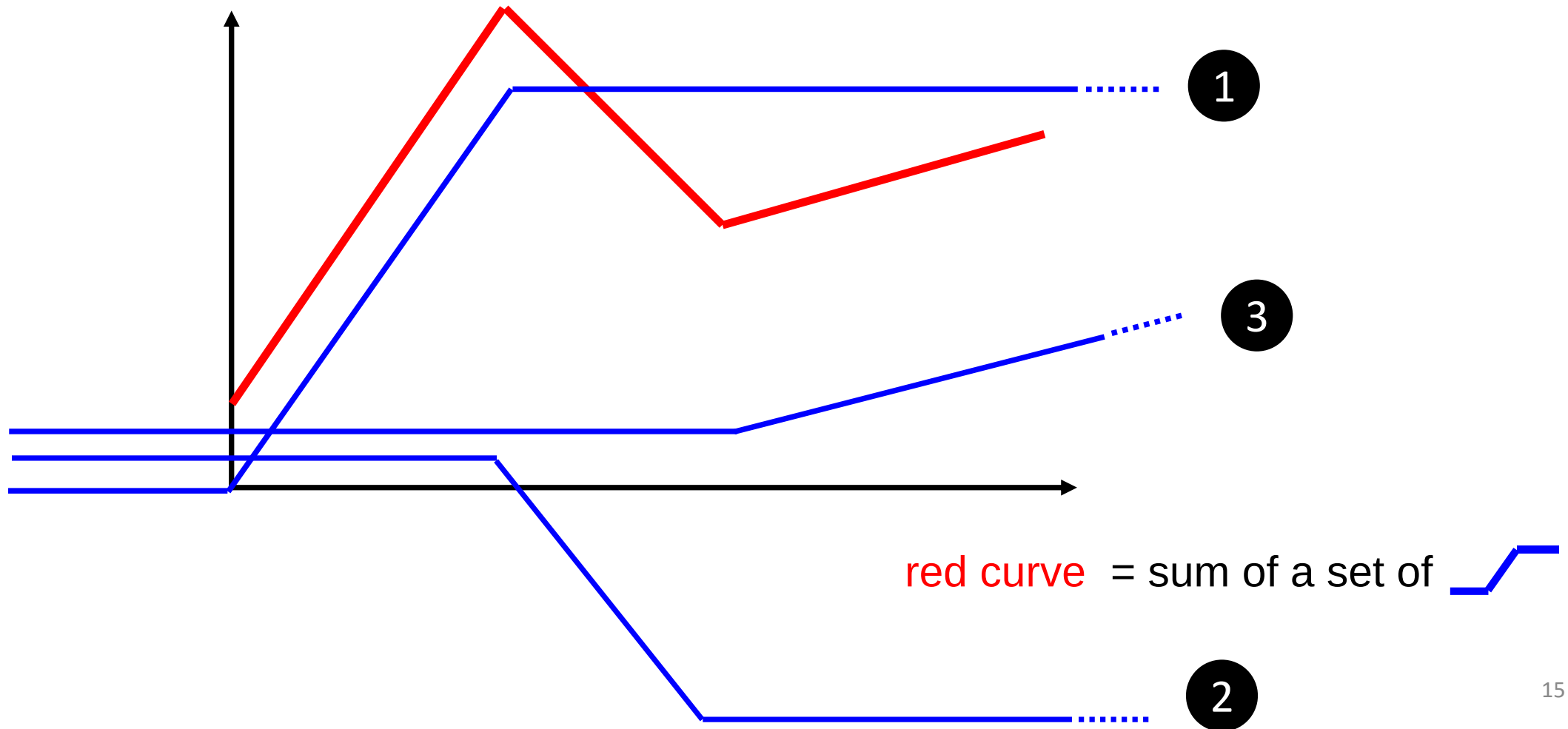


Example of a single neuron



Why does neuron work?

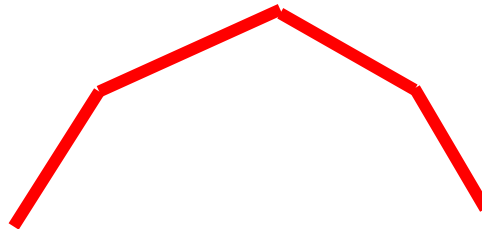
- Given a bunch of neurons with output curve , how to achieve complex decision boundary?



Why does neural work?

- Assemble multiple neuron outputs for complex decision boundary, each neuron takes care of a piece of the entire curve.

red curve = sum of a set of

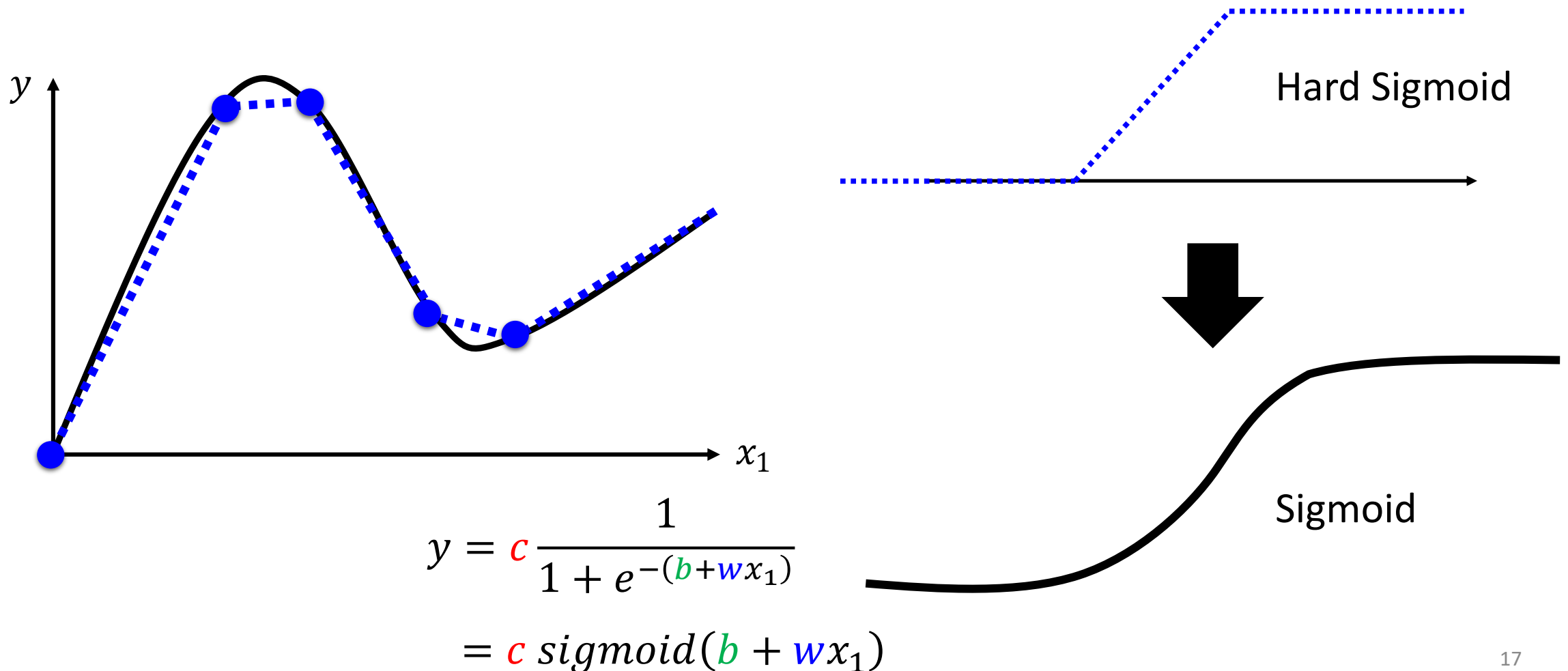


More complex shapes require more



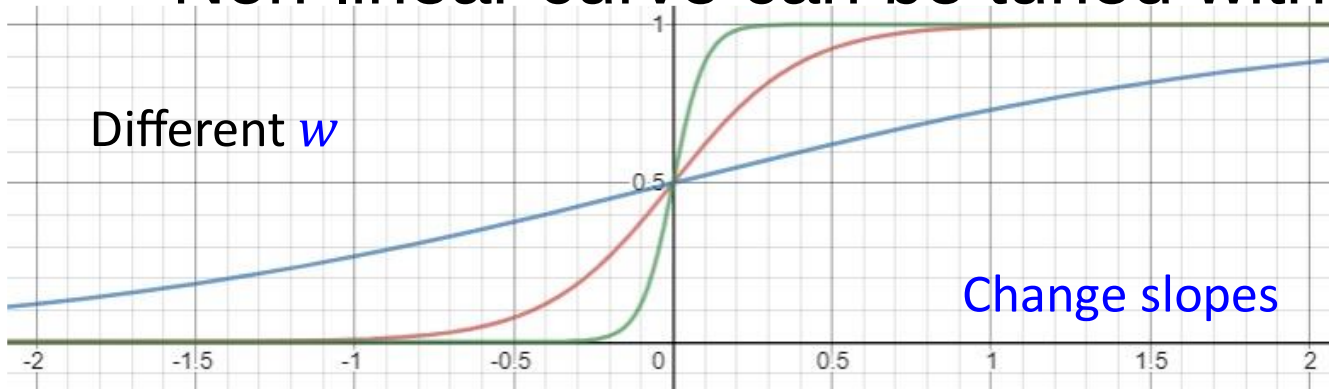
Why does neural network work?

- Using non-linear function, we can introduce smooth curve

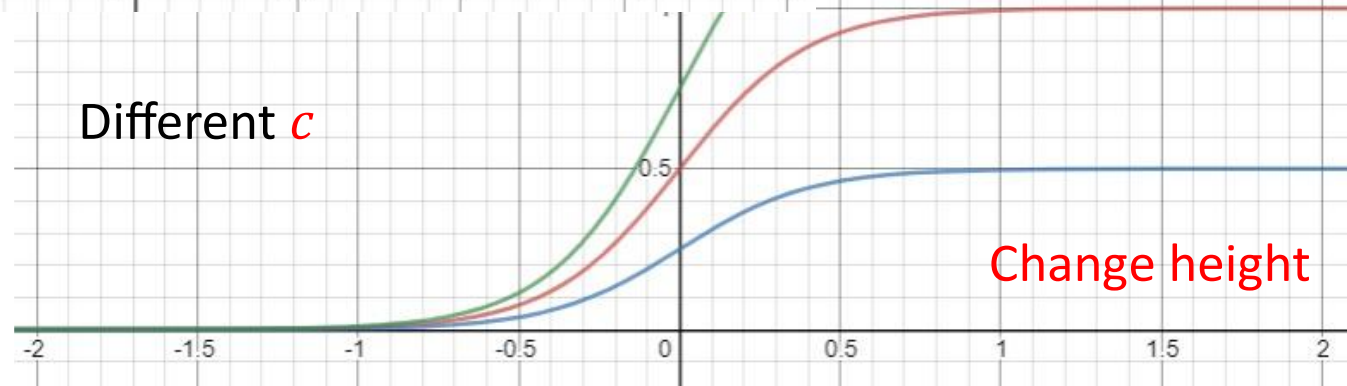
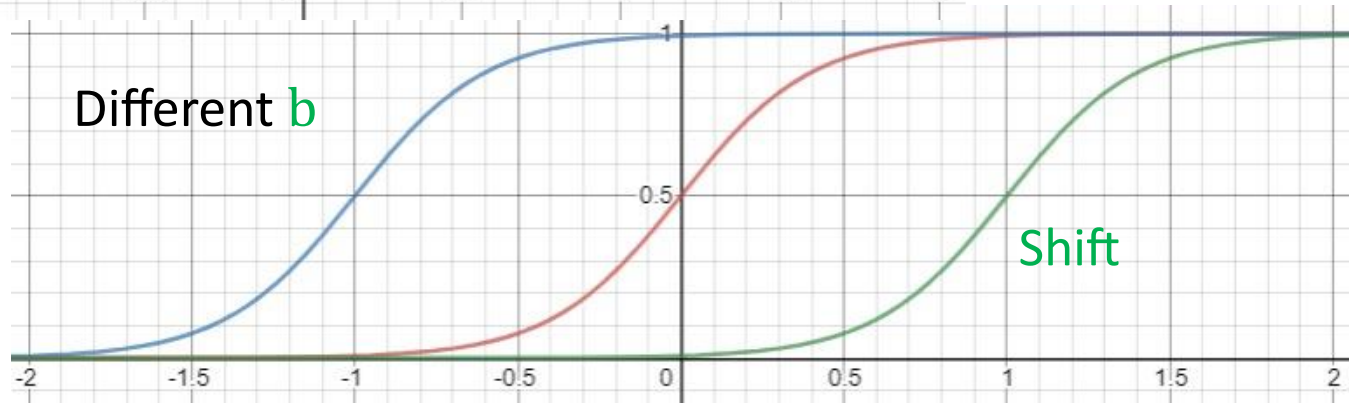


Why does neural network work?

- Non-linear curve can be tuned with network parameters

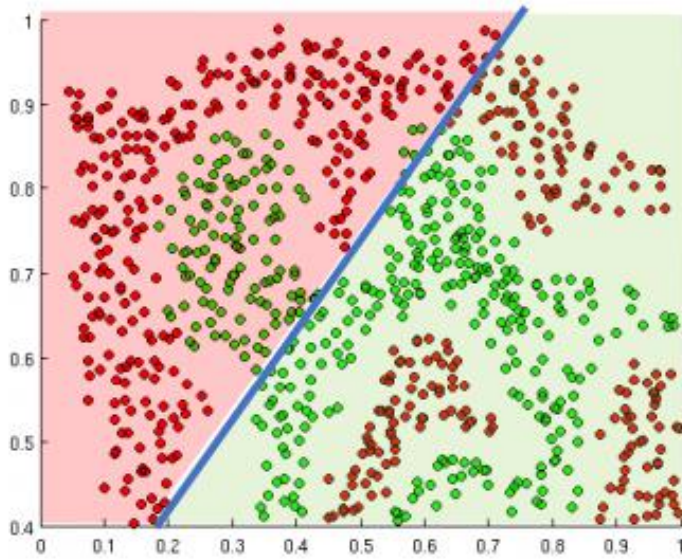


$$y = c \frac{1}{1 + e^{-(b + wx_1)}}$$
$$= c \operatorname{sigmoid}(b + wx_1)$$



Importance of activation function

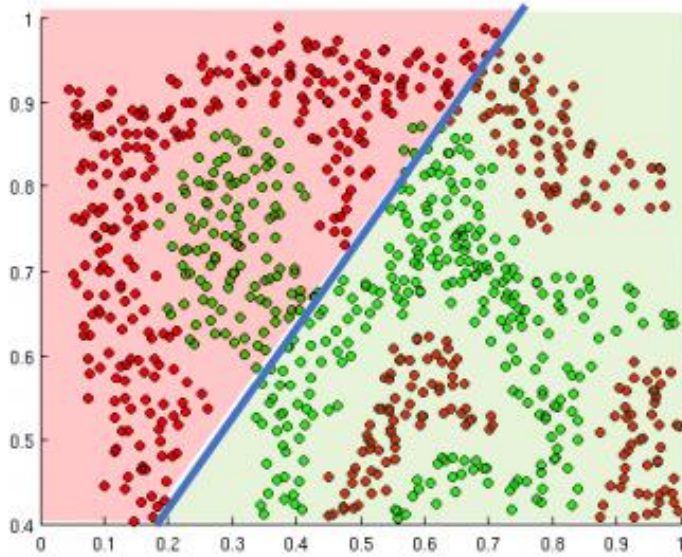
To introduce non-linearities into the decision boundary



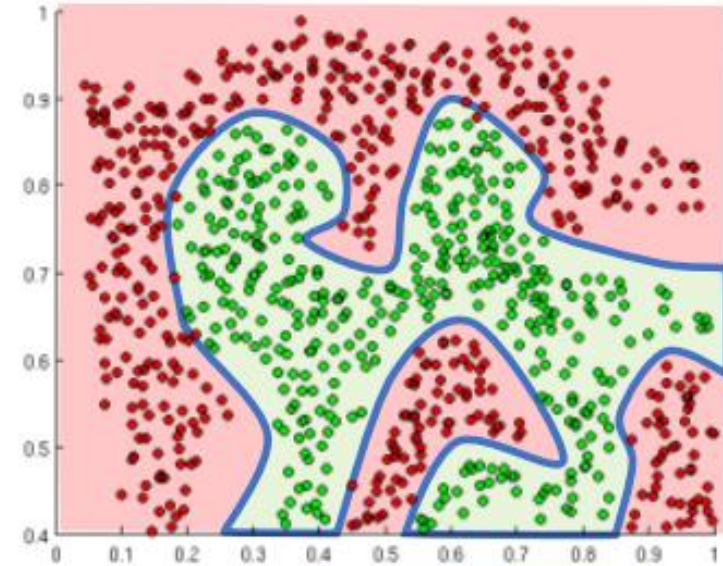
Linear activation functions produce linear decision boundary no matter the network size

Importance of activation function

To introduce non-linearities into the decision boundary



Linear activation functions produce linear decision boundary no matter the network size



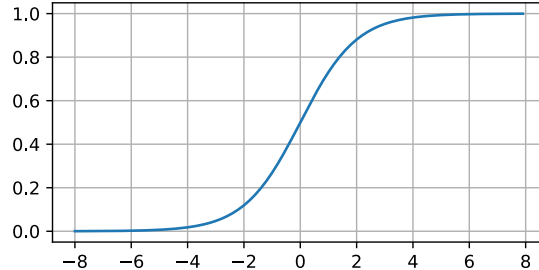
Non-linearities allow us to approximate arbitrarily complex functions

Activation functions

- A mathematical function which adds non-linearity to the network

$$\text{sigmoid}(z) = \frac{1}{1 + \exp(-z)}$$

- Normalized output (0,1)
- Highly compute-intensive
- Not zero-centered

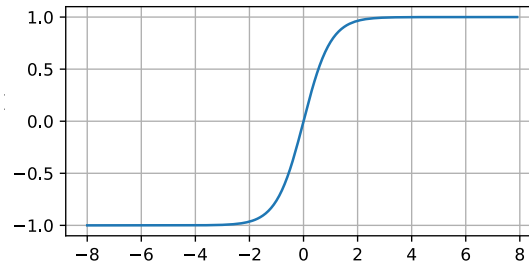


Sigmoid Function

```
>>> m = nn.Sigmoid()
>>> input = torch.randn(2)
>>> output = m(input)
```

$$\tanh(z) = \frac{1 - \exp(-2z)}{1 + \exp(-2z)}$$

- Normalized output (-1,1)
- Highly compute-intensive
- Zero-centered

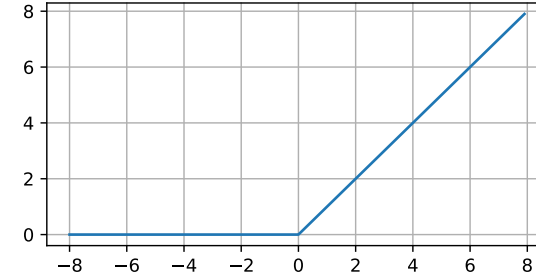


Hyperbolic Tangent (Tanh)

```
>>> m = nn.Tanh()
>>> input = torch.randn(2)
>>> output = m(input)
```

$$\text{ReLU}(z) = \max(z, 0)$$

- Most used, easy-to-compute
- Derivative is not 0 for positive, not zero-centered



Rectified Linear Unit (ReLU)

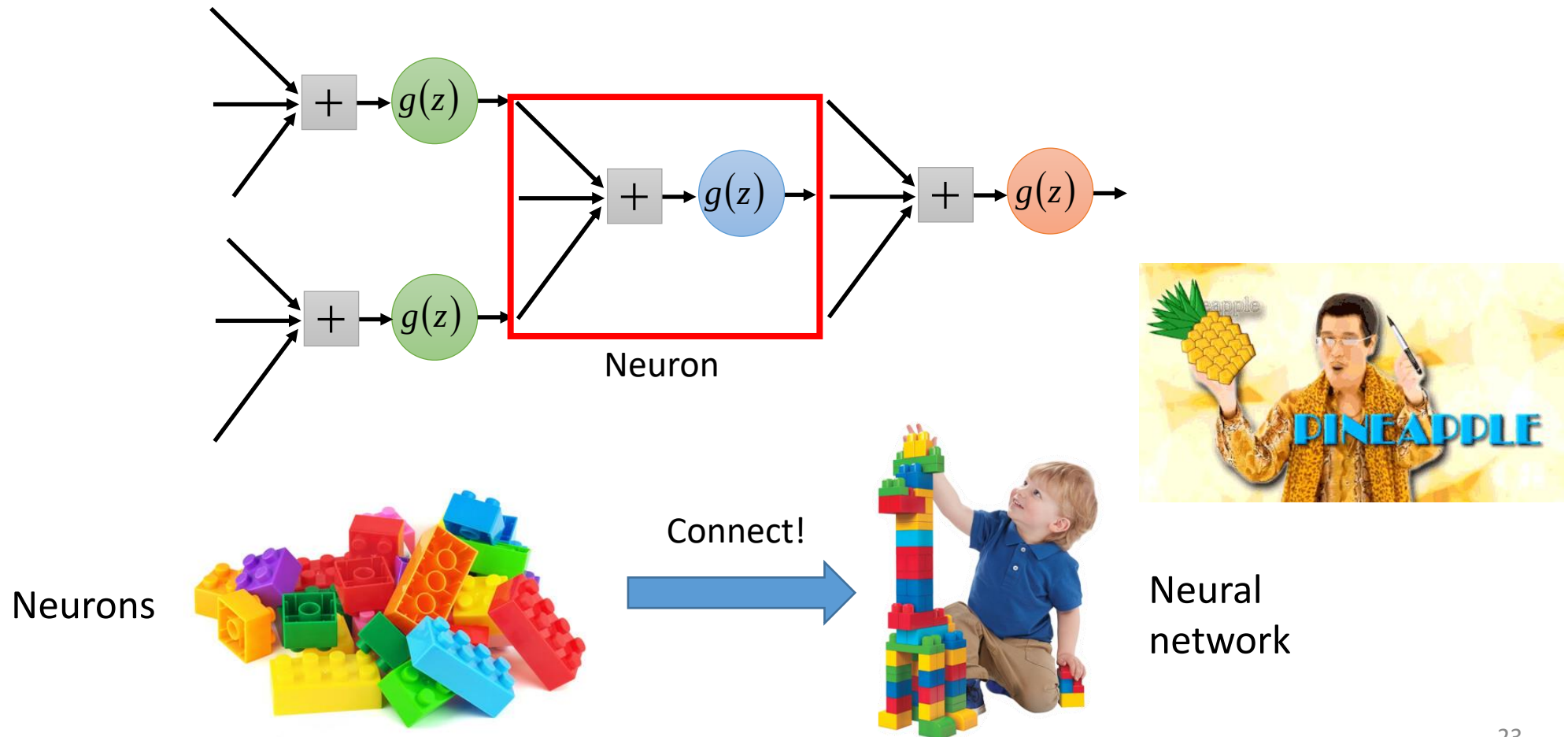
```
>>> m = nn.ReLU()
>>> input = torch.randn(2)
>>> output = m(input)
```

Agenda

- Neural Networks Overview
- Artificial Neuron
- **Multi-Layer Perceptron (MLP) Networks**
- Backpropagation
- Optimization

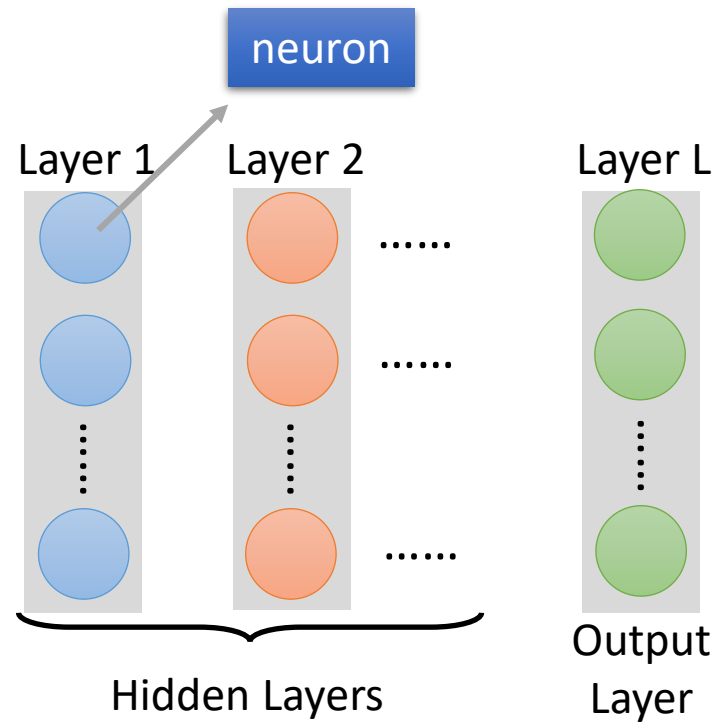
Multi-Layer Perceptron (MLP)

- A **vanilla neural network** - each neuron is fully connected with all the input nodes in the previous layer, and all the output nodes in the next layer



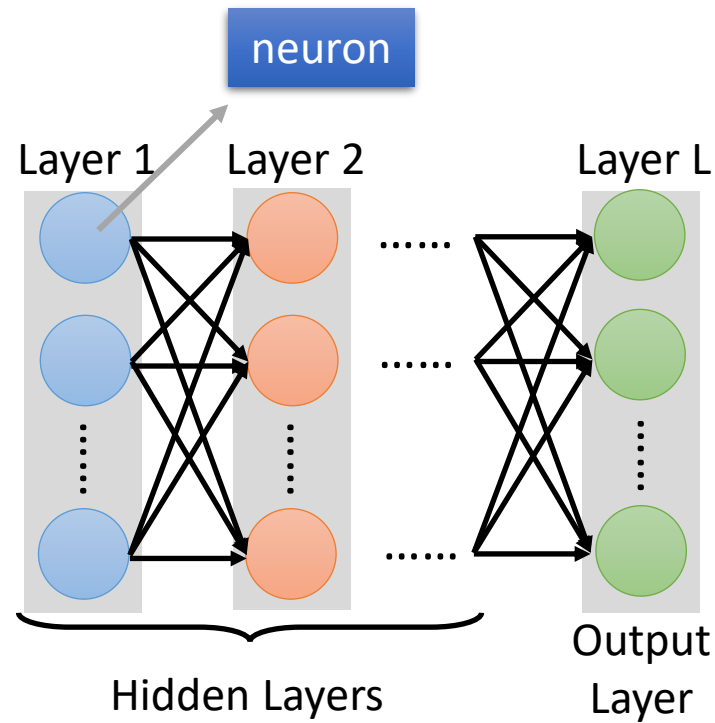
Multi-Layer Perceptron (MLP)

- Hidden and output layer are a collection of neurons



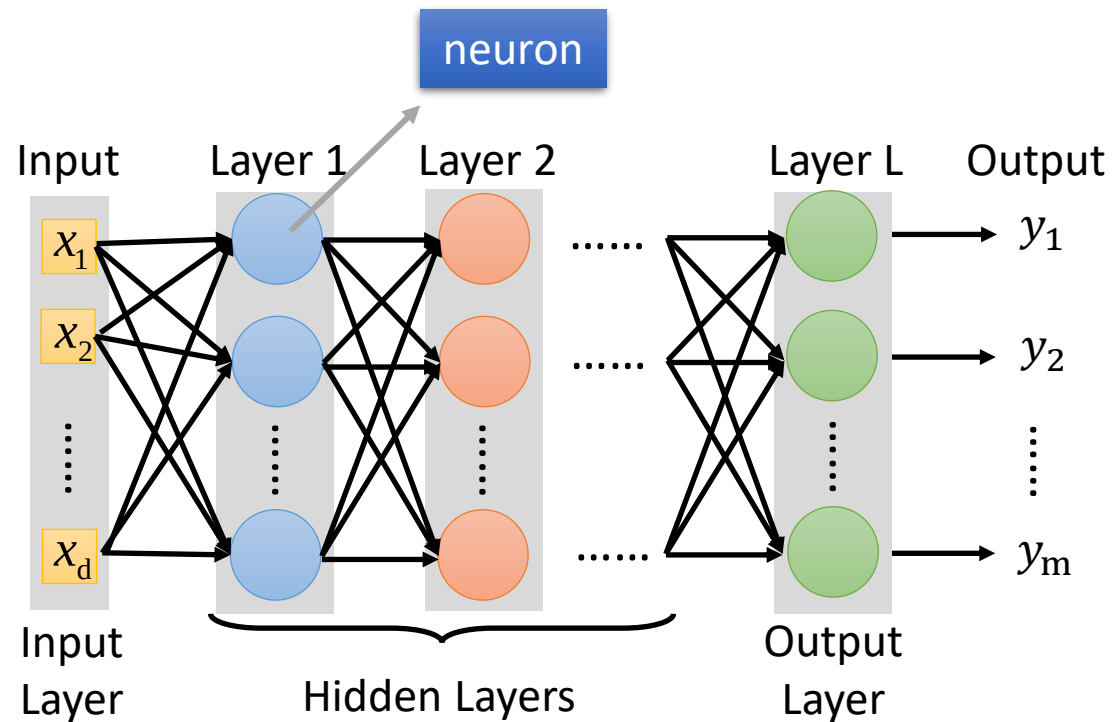
Multi-Layer Perceptron (MLP)

- Hidden and output layer are a collection of neurons
- Each layers is fully connected with previous and next layer



Multi-Layer Perceptron (MLP)

- Hidden and output layer are a collection of neurons
- Each layers is fully connected with previous and next layer
- Information are processed from end to end



A two-layer example:

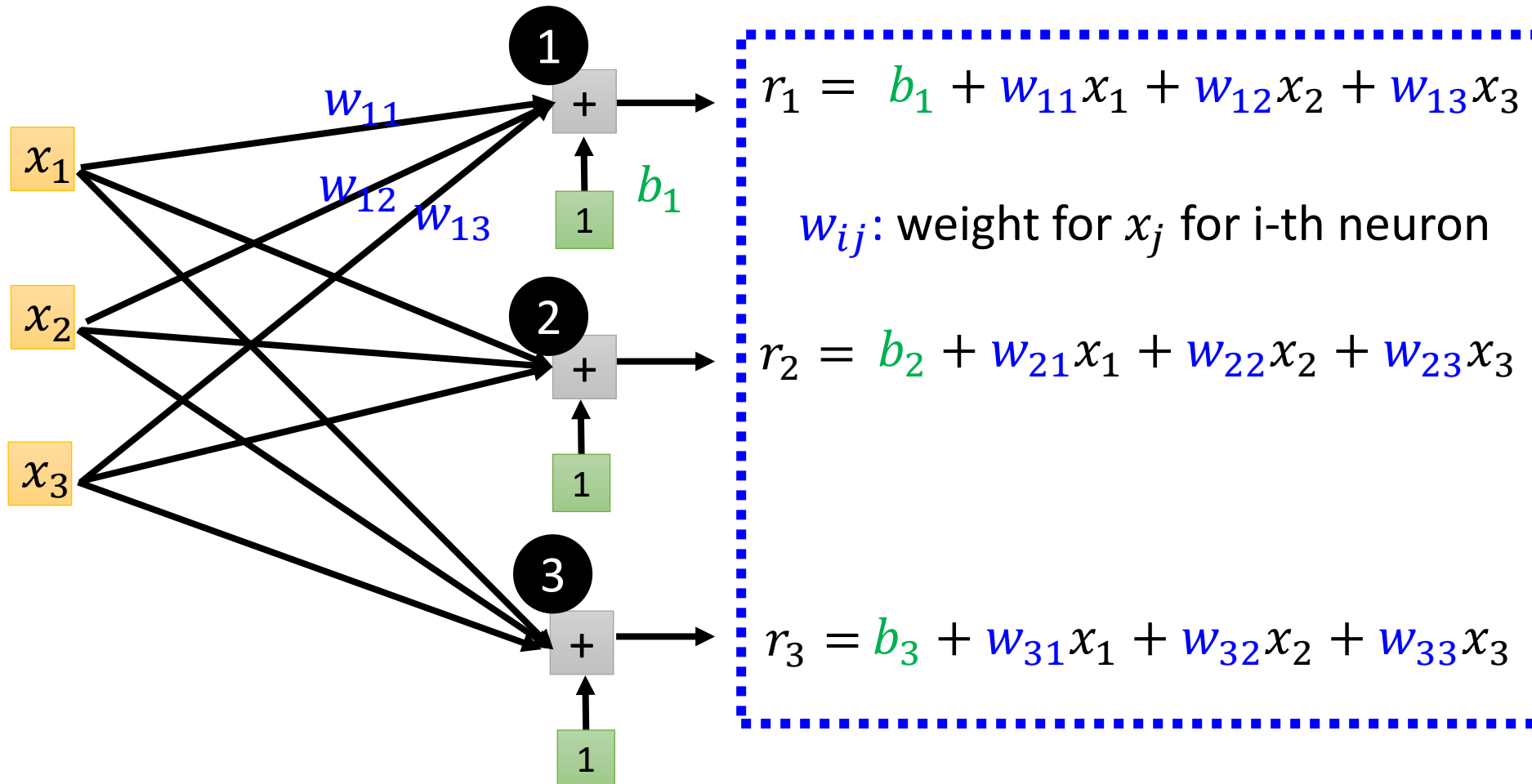
$j: 1,2,3$

no. of features

$$y = b + \sum_i \textcolor{red}{c}_i \text{sigmoid} \left(\textcolor{green}{b}_i + \sum_j \textcolor{blue}{w}_{ij} x_j \right)$$

$i: 1,2,3$

no. of neurons



A two-layer example:

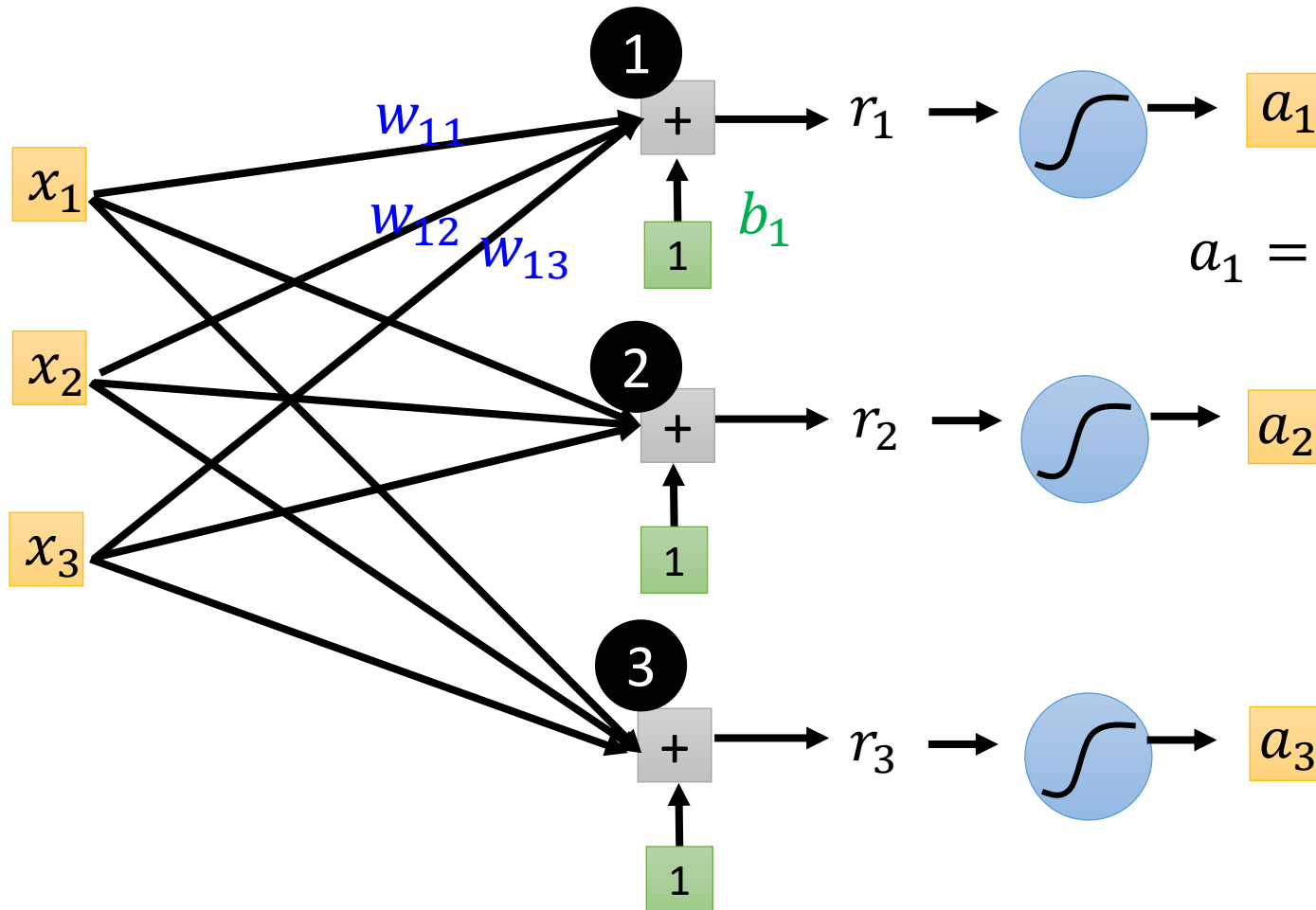
$$y = b + \sum_i c_i \text{sigmoid} \left(b_i + \sum_j w_{ij} x_j \right)$$

$j: 1,2,3$

no. of features

$i: 1,2,3$

no. of neurons



$$a_1 = \text{sigmoid}(r_1) = \frac{1}{1 + e^{-r_1}}$$

A two-layer example:

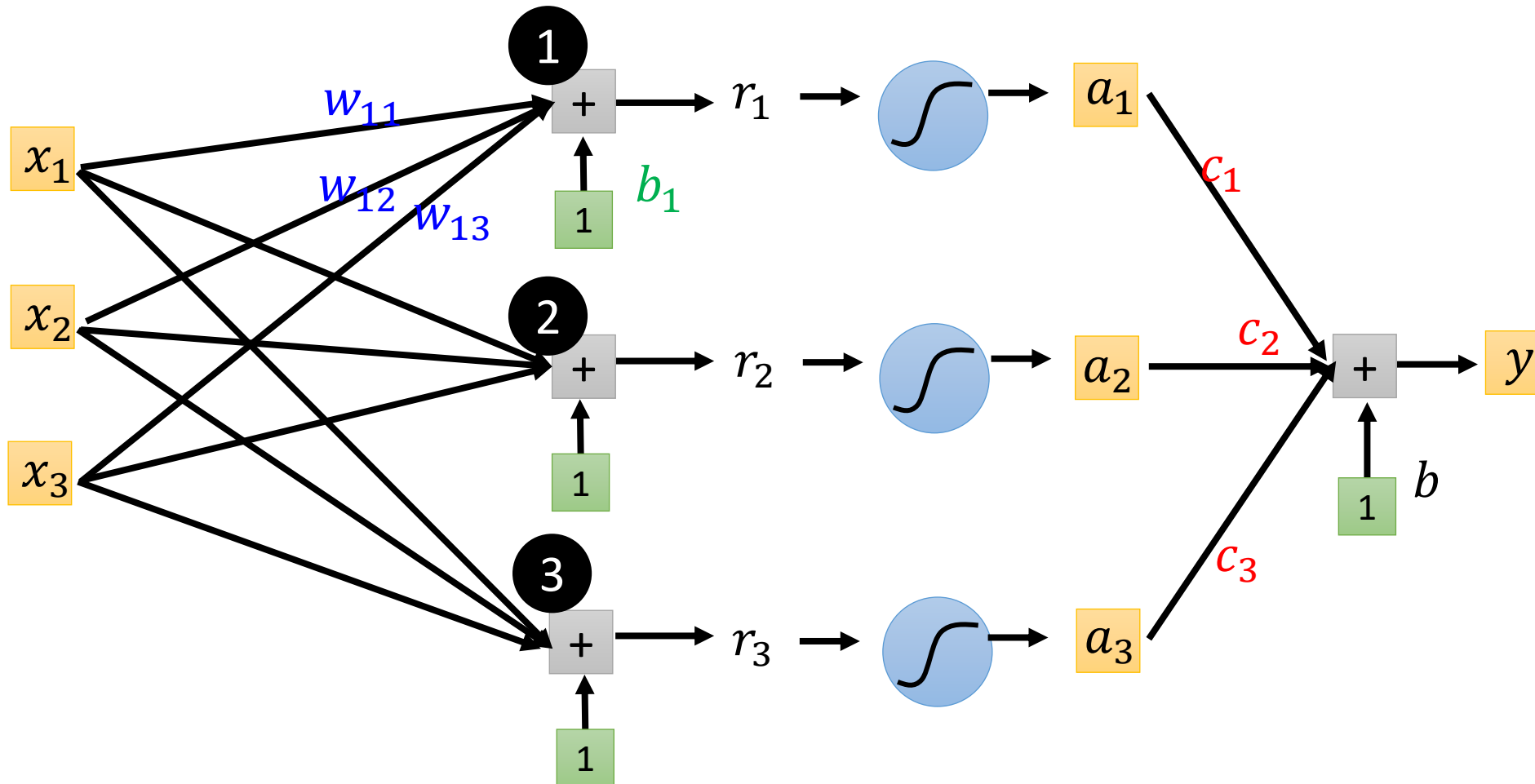
$$y = b + \sum_i c_i \operatorname{sigmoid} \left(b_i + \sum_j w_{ij} x_j \right)$$

$j: 1,2,3$

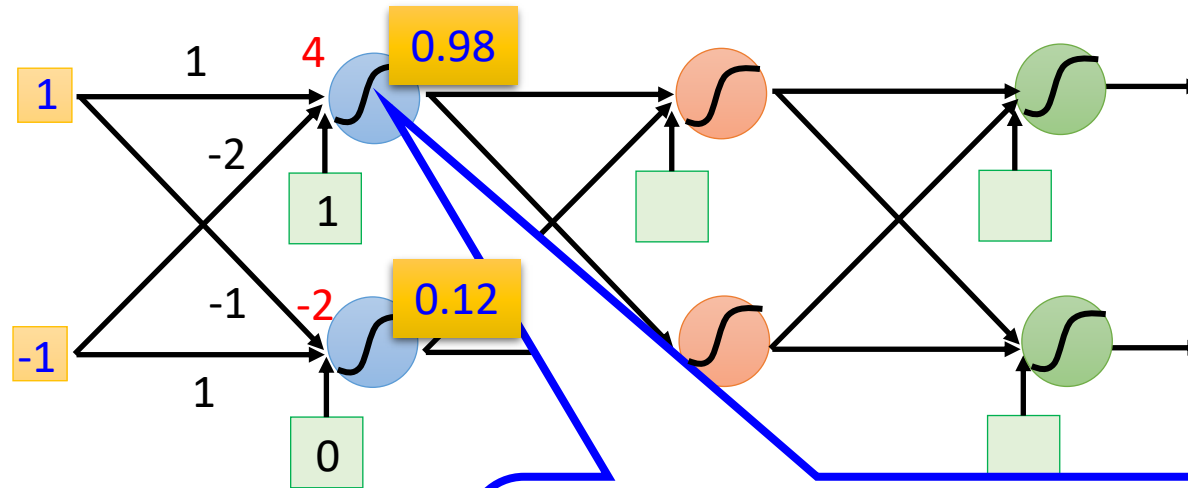
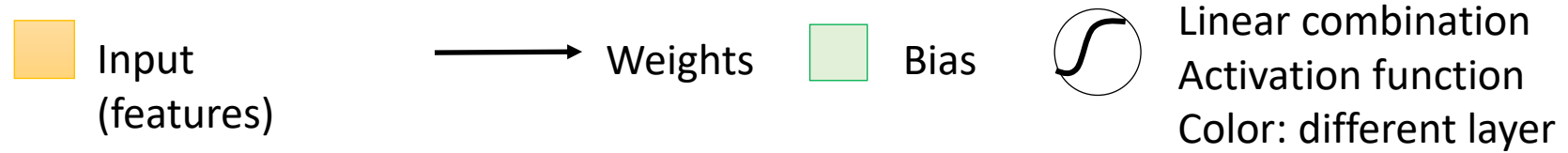
no. of features

$i: 1,2,3$

no. of neurons

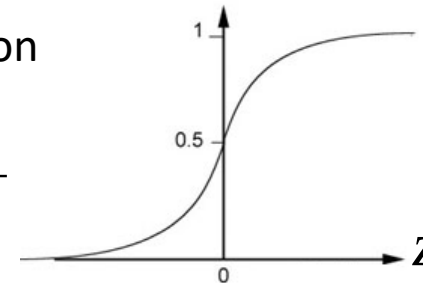


Forward propagation

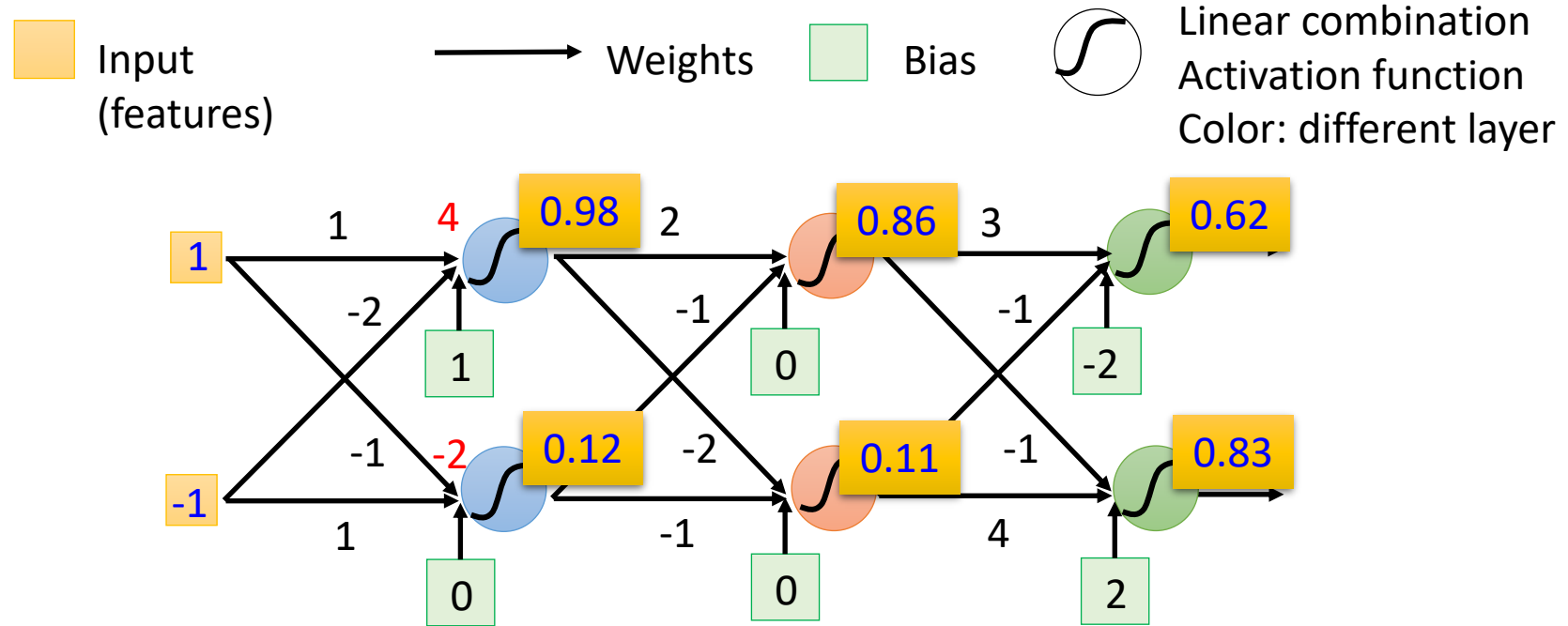


Sigmoid Function

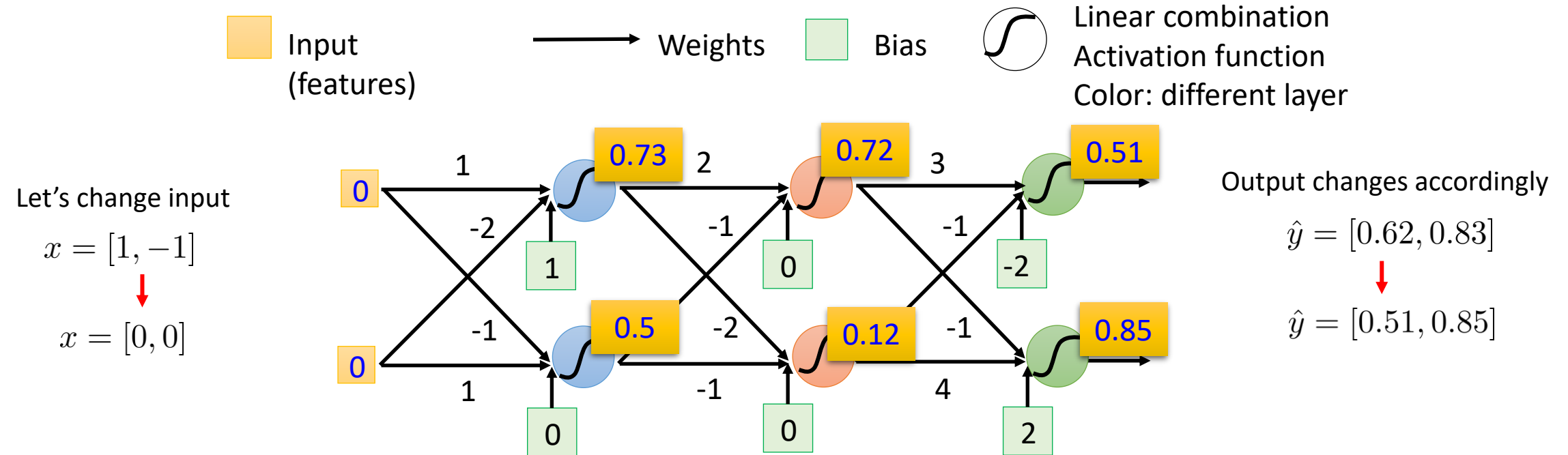
$$g(z) = \frac{1}{1 + e^{-z}}$$



Forward propagation



Forward propagation



Input vector

Output vector

$$f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix}$$

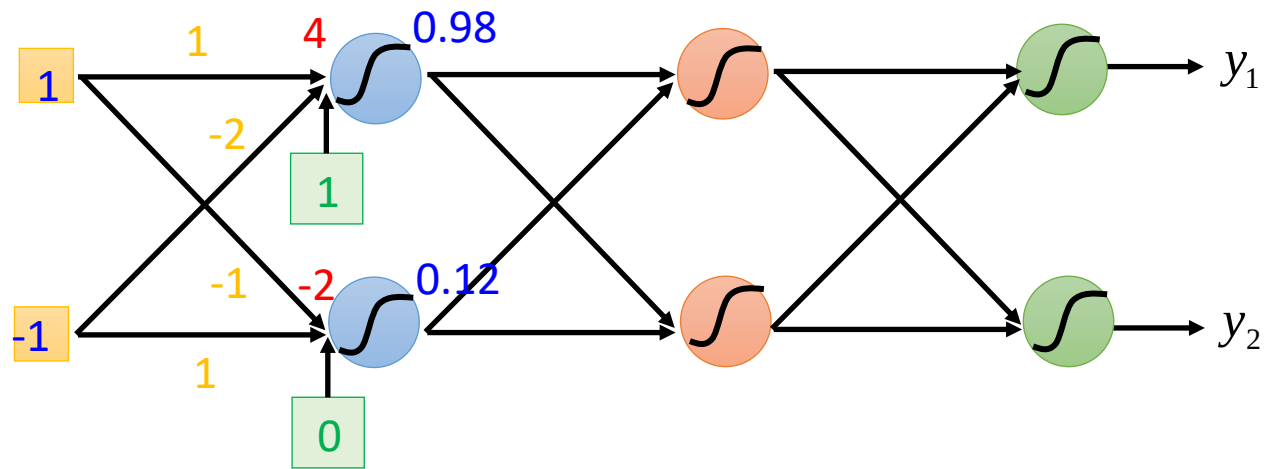
$$f\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0.51 \\ 0.85 \end{bmatrix}$$

Given a network structure with fixed parameters

We define a function!

Vectorization

- A computation processes the entire layer (all the neurons)



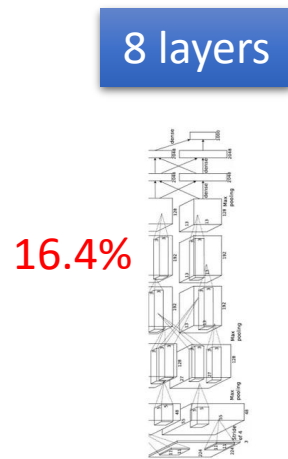
$$g\left(\underbrace{\begin{bmatrix} 1 & -2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix}}_{\begin{bmatrix} 4 \\ -2 \end{bmatrix}} \right) = \begin{bmatrix} 0.98 \\ 0.12 \end{bmatrix}$$

Weights of 1st hidden layer Input of 1st layer Bias of 1st layer Output of 1st layer

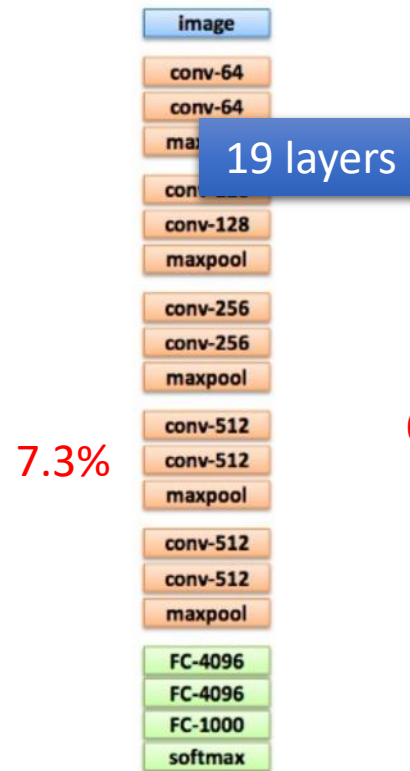
More hidden layers (deep model)

Benefits

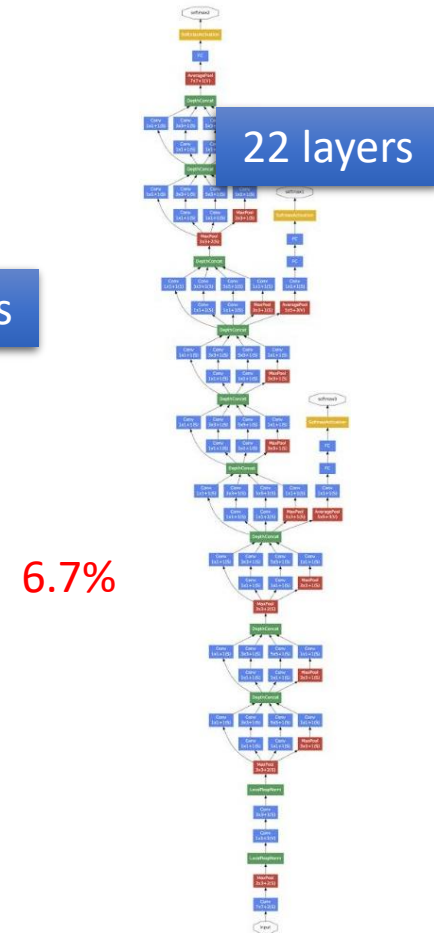
- More complex function
- Better performance



AlexNet (2012)



VGG (2014)



GoogleNet (2014)

Deeper: Number of hidden layers

Better: Error rate (misclassification)



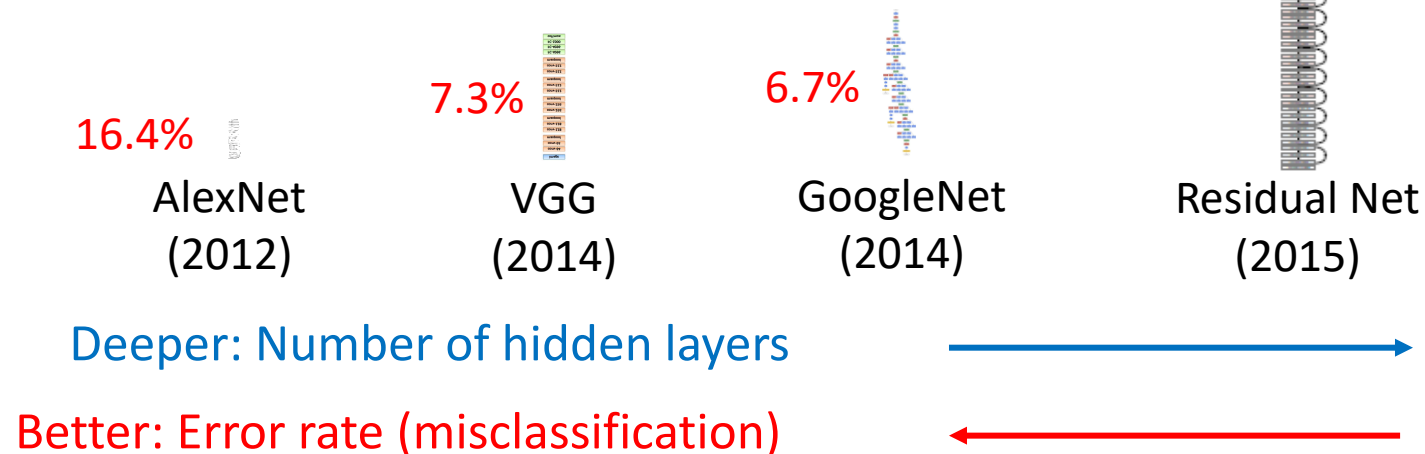
More hidden layers (deep model)

Benefits

- More complex function
- Better performance

Issues

- **Gradient vanish** (gradient gets too small to update parameters)
- **Overfitting** by having model too complex
- **Hard to converge** by optimizing too many parameters



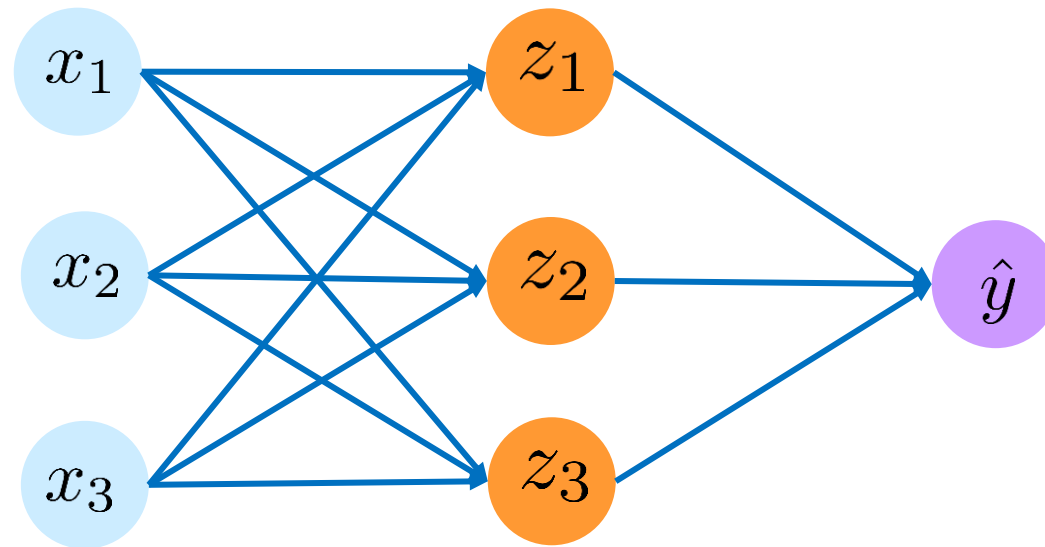
Agenda

- Neural Networks Overview
- Artificial Neuron
- Multi-Layer Perceptron (MLP) Networks
- **Backpropagation**
- Optimization

Learning neural networks

Example: Is this email spam or not

	x_1	x_2	x_3	y
feature/label	"dating"	"learning"	"cash"	Spam?
email 1	1	0	1	1



Inputs: $x^{(i)} = [1, 0, 1]$

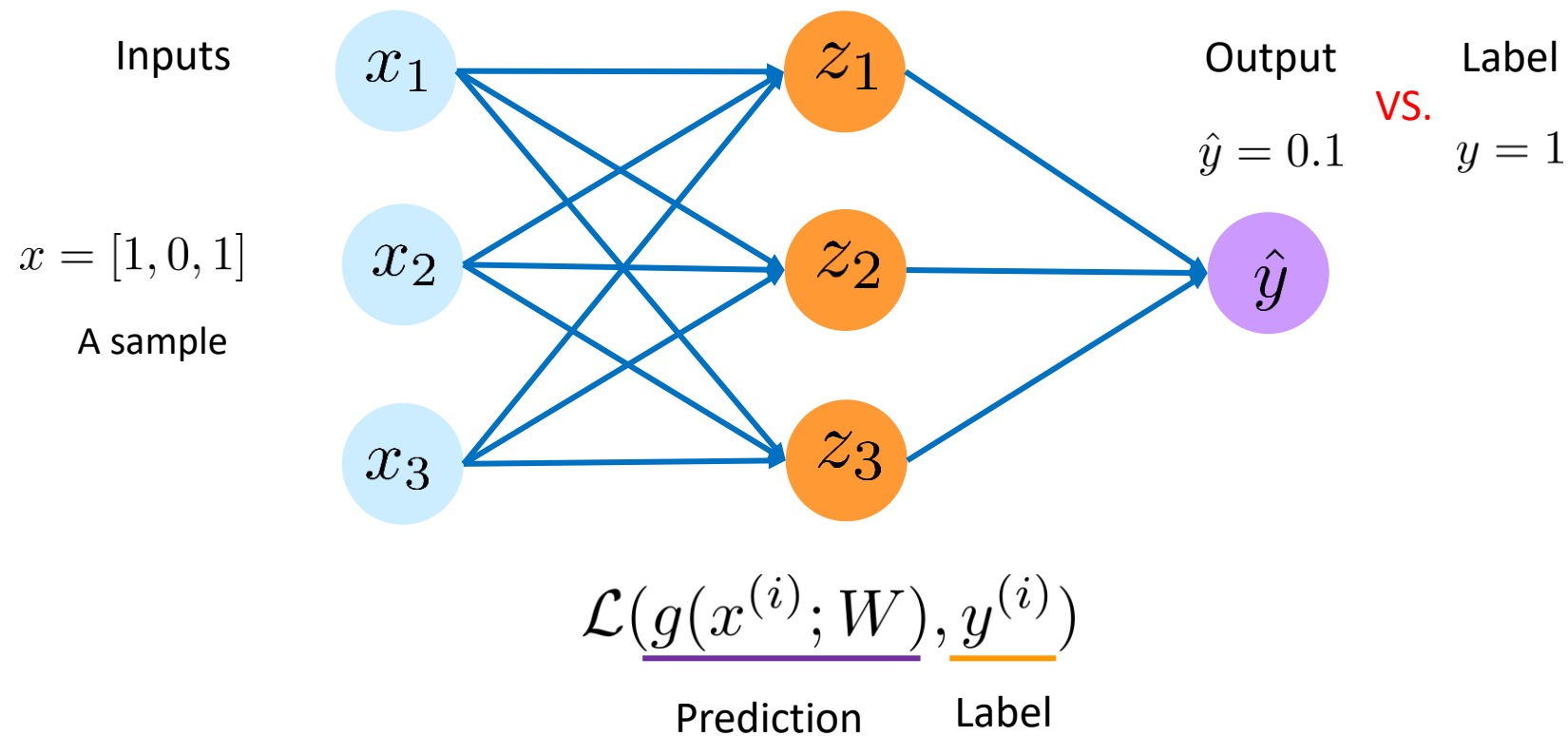
Hidden layer
(3 neurons)

Prediction: $\hat{y}^{(i)} = 0.1$
Label: $y^{(i)} = 1$

Measuring loss

Loss of the network

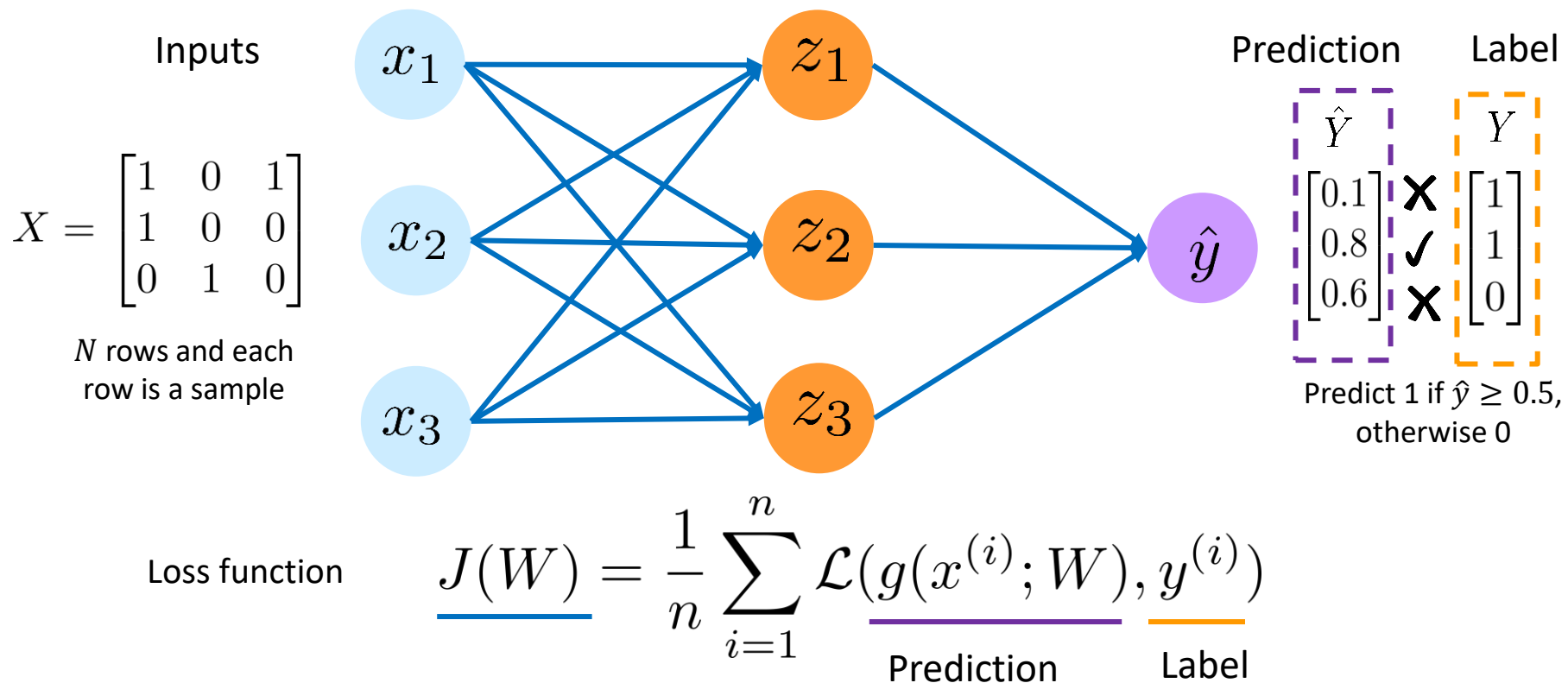
- Measures the cost incurred at output (prediction)



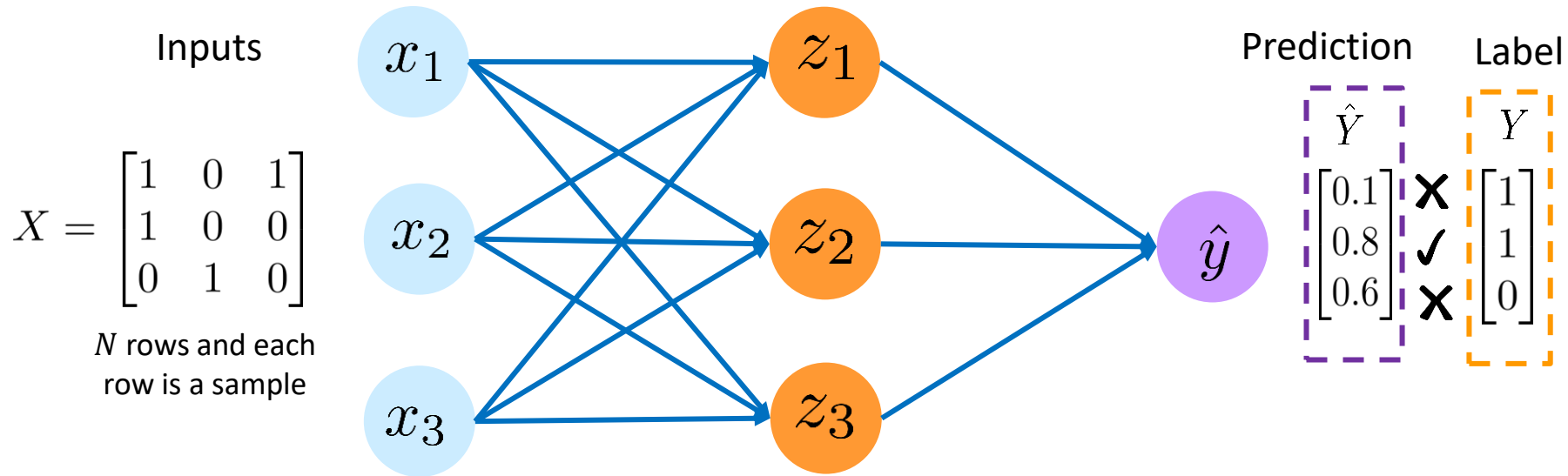
Measuring loss

Loss of the network

- Measures the cost incurred at output (prediction)
- Collected by each sample, aggregated over all data



Cross entropy loss



- Measures the loss for binary classification where output is a probability between 0 and 1

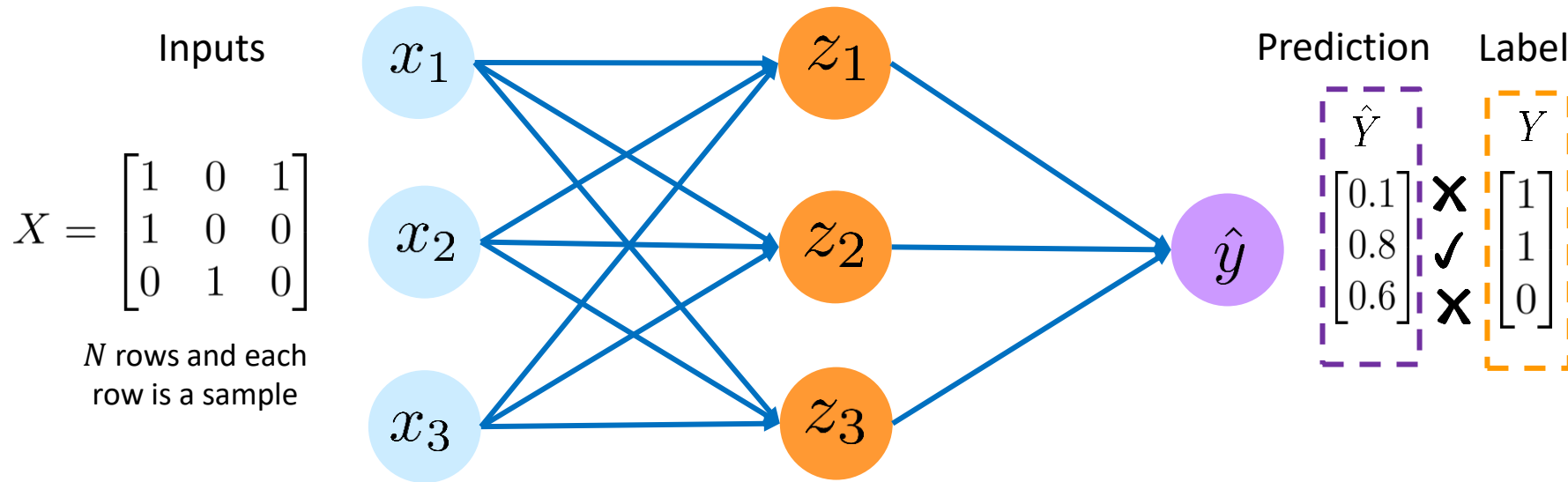
$$J(W) = -\frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Label}} \underbrace{\log(g(x^{(i)}; W))}_{\text{Prediction}} + \underbrace{(1 - y^{(i)})}_{\text{Label}} \underbrace{\log(1 - g(x^{(i)}; W))}_{\text{Prediction}}$$

Sample with positive label Sample with negative label

PyTorch

```
loss = torch.nn.BCEWithLogitsLoss()
output = loss(predicted, y, reduction='mean')
```


Mean Squared Error (MSE) loss



- Measures the loss for regression where output is a continuous real number

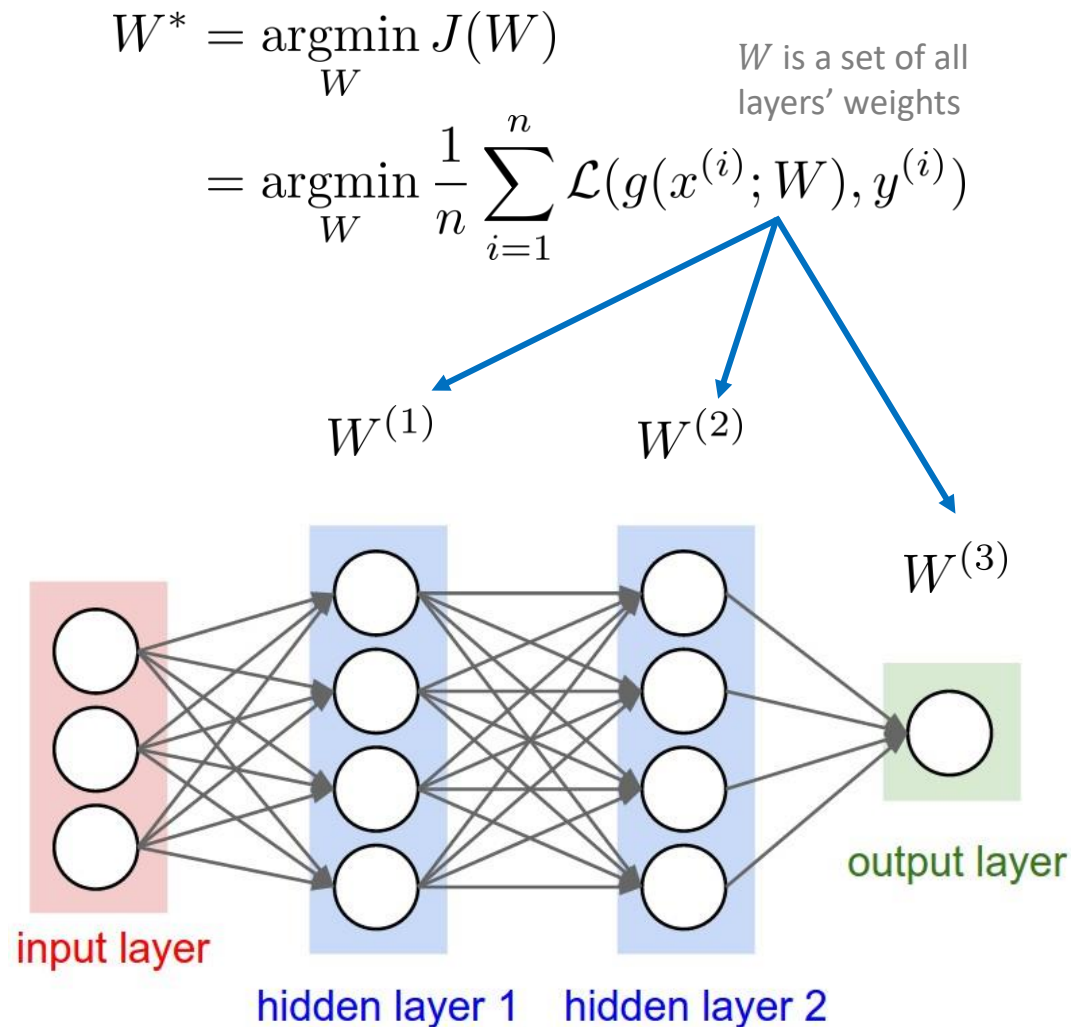
$$J(W) = \frac{1}{n} \sum_{i=1}^n (\underbrace{y^{(i)}}_{\text{Label}} - \underbrace{g(x^{(i)}; W)}_{\text{Prediction}})^2$$

PyTorch

```
loss = torch.nn.MSELoss()  
output = loss(predicted, y, reduction='mean')
```

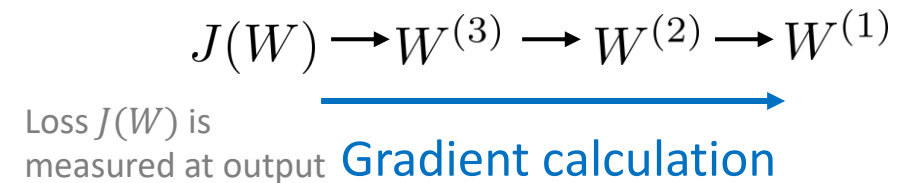
Update parameters to optimize loss

- **Goal:** find the network weights that achieve the lowest loss



Weights in every layer need to be optimized

- Loss is calculation at output
- We need to update parameters to lower loss
- How do we decide gradient for each parameter?
- **Backpropagation!**

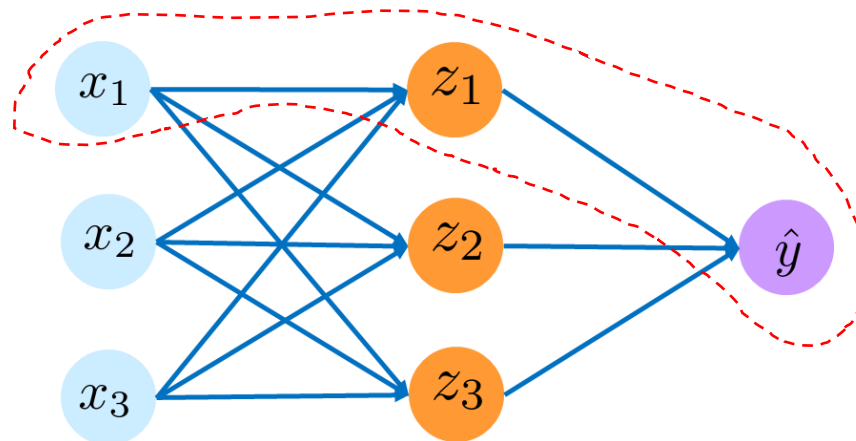
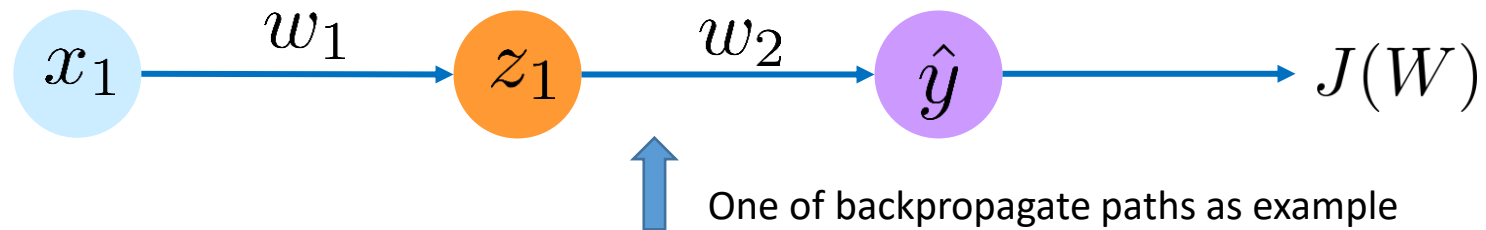


Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$\begin{array}{l} ? \quad \Delta w_1 \rightarrow \Delta J(W) \\ \quad \Delta w_2 \rightarrow \Delta J(W) \end{array}$$

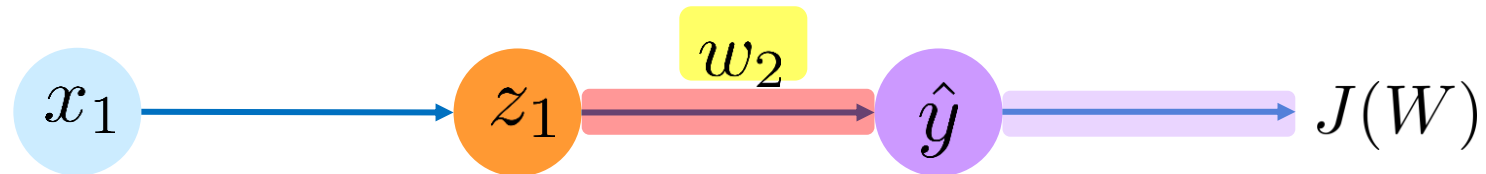


Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$? \quad \Delta w_2 \rightarrow \Delta J(W)$$



$$\frac{\partial J(W)}{\partial w_2} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} \times \underbrace{\frac{\partial \hat{y}}{\partial w_2}}_{\text{red}}$$

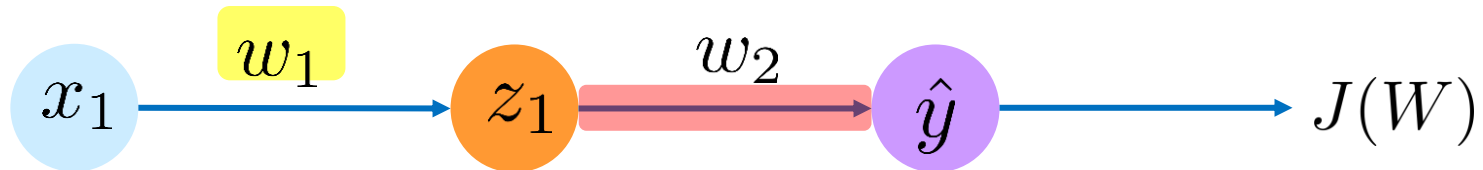
Chain rule: from back to front

Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$? \quad \Delta w_1 \rightarrow \Delta J(W)$$



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w_1}$$

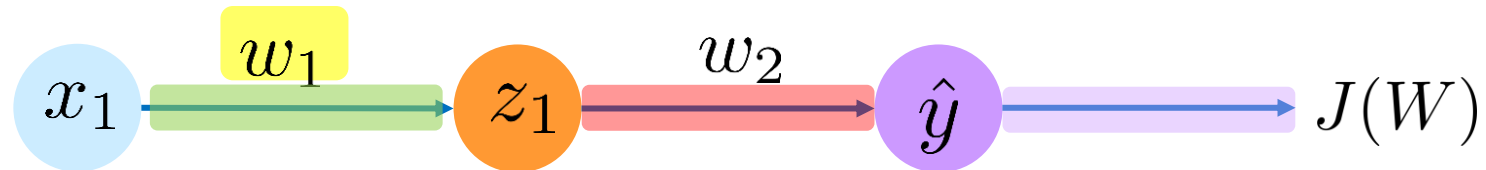
Apply chain rule

Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$? \quad \Delta w_1 \rightarrow \Delta J(W)$$



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} \times \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} \times \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{green}}$$

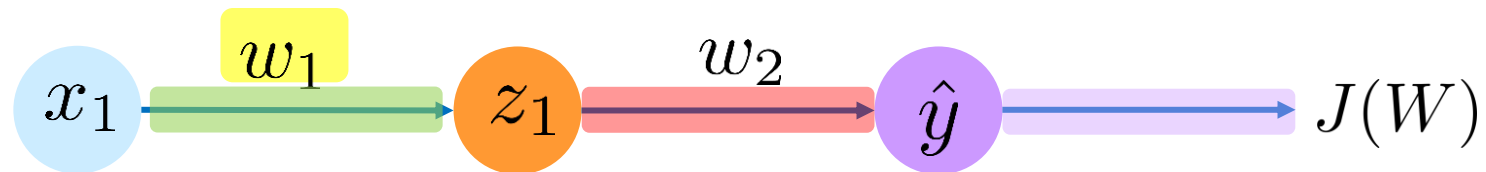
Chain rule: one more layer backward

Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$? \quad \Delta w_1 \rightarrow \Delta J(W)$$

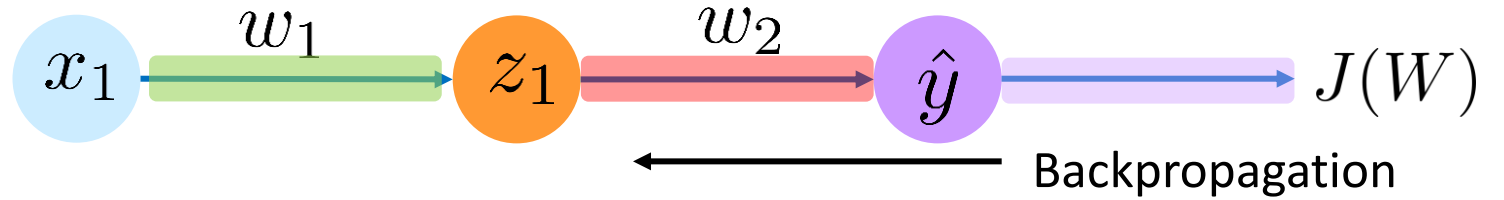


$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} \times \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} \times \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{green}}$$

Repeat this “message passing” for **every weight** in each layer using the gradients from **afterward layers**, until all the layers have been updated by loss.

Gradients by partial derivative

- Cross Entropy Loss



$$J(W) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \log(\hat{y}) + (1 - y^{(i)}) \log(1 - \hat{y}) \quad \text{Each neuron } f(x) = \frac{1}{1 + e^{-w^\top x}}$$

Partial derivative of **loss** respect to $\hat{y}$

$$\begin{aligned} \frac{\partial J(W)}{\partial \hat{y}} &= -(y \times \frac{1}{\hat{y}} + (1 - y) \times \frac{1}{1 - \hat{y}} \times -1) \\ &= \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \end{aligned}$$

Gradient for w_2

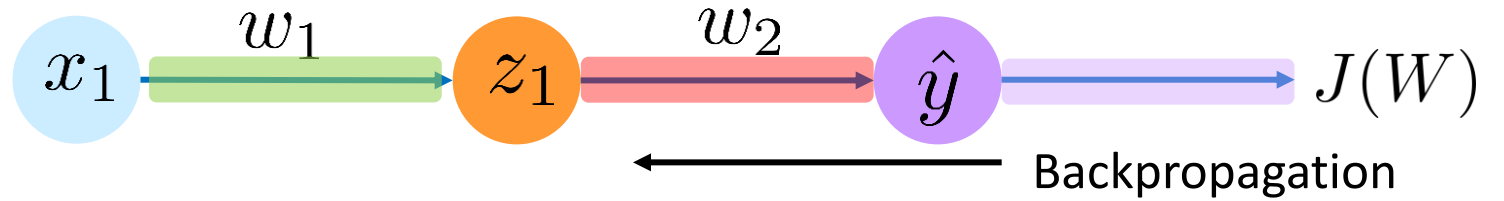
$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w_2}$$

Partial derivative of $\hat{y}$ respect to a weight w_2

$$\begin{aligned} \frac{\partial \hat{y}}{\partial w_2} &= -1 \times (1 + e^{-w_2^\top z_1})^{-2} \times e^{-w_2^\top z_1} \times -z_1 \\ &= \frac{1}{1 + e^{-w_2^\top z_1}} \times \frac{e^{-w_2^\top z_1}}{1 + e^{-w_2^\top z_1}} \times z_1 \\ &= \hat{y}(1 - \hat{y})z_1 \end{aligned}$$

Gradients by partial derivative

- Cross Entropy Loss



$$J(W) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \log(\hat{y}) + (1 - y^{(i)}) \log(1 - \hat{y}) \quad \text{Each neuron } f(x) = \frac{1}{1 + e^{-w^\top x}}$$

Partial derivative of **loss** respect to $\hat{y}$

$$\begin{aligned} \frac{\partial J(W)}{\partial \hat{y}} &= -(y \times \frac{1}{\hat{y}} + (1 - y) \times \frac{1}{1 - \hat{y}} \times -1) \\ &= \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \end{aligned}$$

Partial derivative of $\hat{y}$ respect to a weight w_2

$$\begin{aligned} \frac{\partial \hat{y}}{\partial w_2} &= -1 \times (1 + e^{-w_2^\top z_1})^{-2} \times e^{-w_2^\top z_1} \times -z_1 \\ &= \frac{1}{1 + e^{-w_2^\top z_1}} \times \frac{e^{-w_2^\top z_1}}{1 + e^{-w_2^\top z_1}} \times z_1 \\ &= \hat{y}(1 - \hat{y})z_1 \end{aligned}$$

Partial derivative of $\hat{y}$ respect to a weight z_1

$$\begin{aligned} \frac{\partial \hat{y}}{\partial z_1} &= -1 \times (1 + e^{-w_2^\top z_1})^{-2} \times e^{-w_2^\top z_1} \times -w_2 \\ &= \hat{y}(1 - \hat{y})w_2 \end{aligned}$$

Partial derivative of $\hat{y}$ respect to a weight z_1

$$\begin{aligned} \frac{\partial z_1}{\partial w_1} &= -1 \times (1 + e^{-w_1^\top x_1})^{-2} \times e^{-w_1^\top x_1} \times -x_1 \\ &= z_1(1 - z_1)x_1 \end{aligned}$$

Gradient for w_2

$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w_2}$$

Gradient for w_1

$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z_1} \times \frac{\partial z_1}{\partial w_1}$$

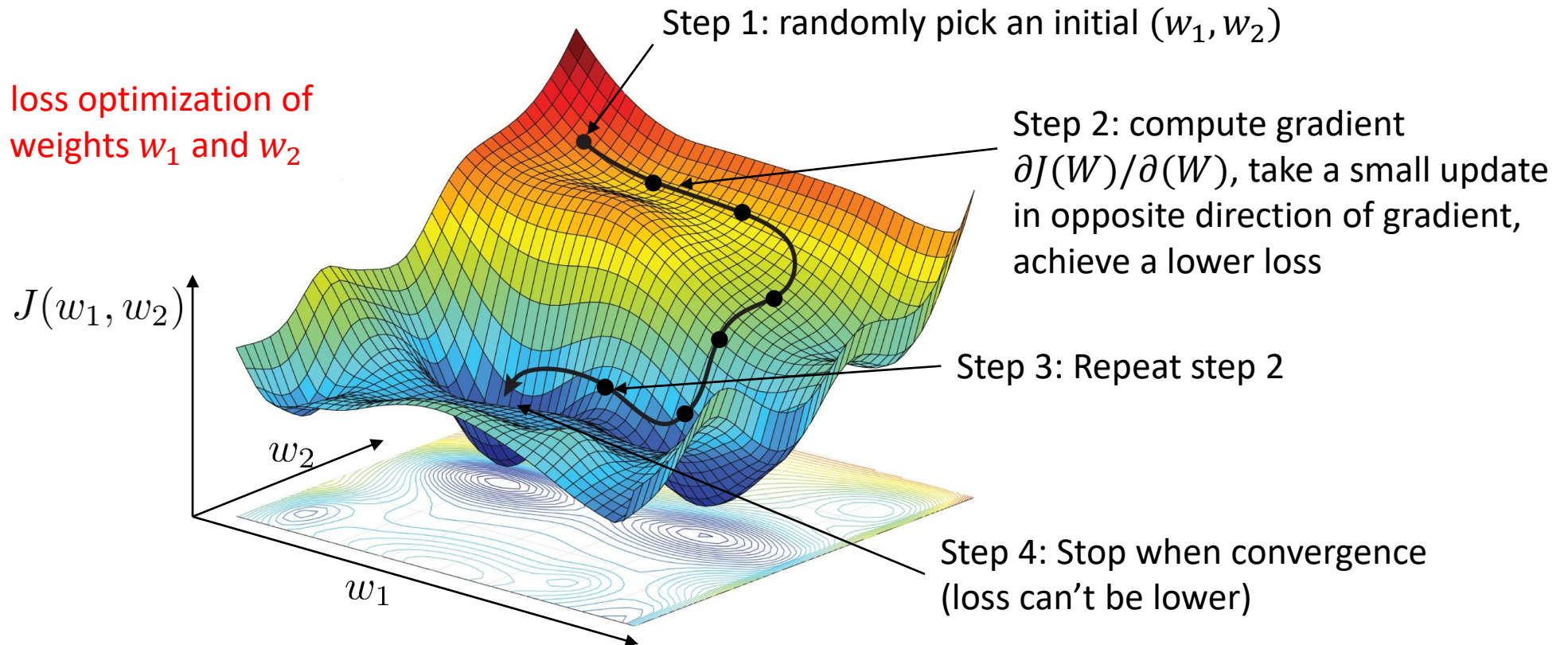
Agenda

- Neural Networks Overview
- Artificial Neuron
- Multi-Layer Perceptron (MLP) Networks
- Backpropagation
- Optimization

Loss optimization

- **Goal:** find the network weights that achieve the lowest loss

Example of a loss optimization of two network weights w_1 and w_2



Distribution of the loss

Red: loss is high

Blue: loss is low

Gradient descent

Gradient Descent (GD)

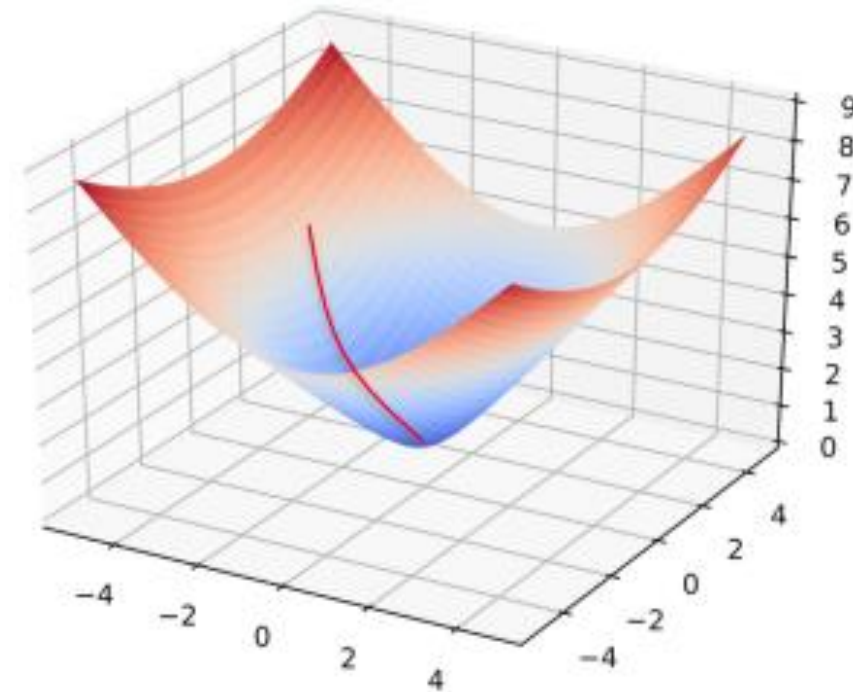
1. Initialize weights $W \leftarrow W^{(0)}$
2. While not converged do:
3. Compute gradient $\nabla_W J(W) = \frac{\partial J(W)}{\partial (W)}$
4. Update weights
 $W \leftarrow W - \alpha \cdot \nabla_W J(W)$
5. Return W

Benefit

- Reliable on optimizing networks in the direction of the optimal

Drawback

- Large computation by go over the the entire data in each iteration



Convergence can be slow when data size increases!

Stochastic gradient descent

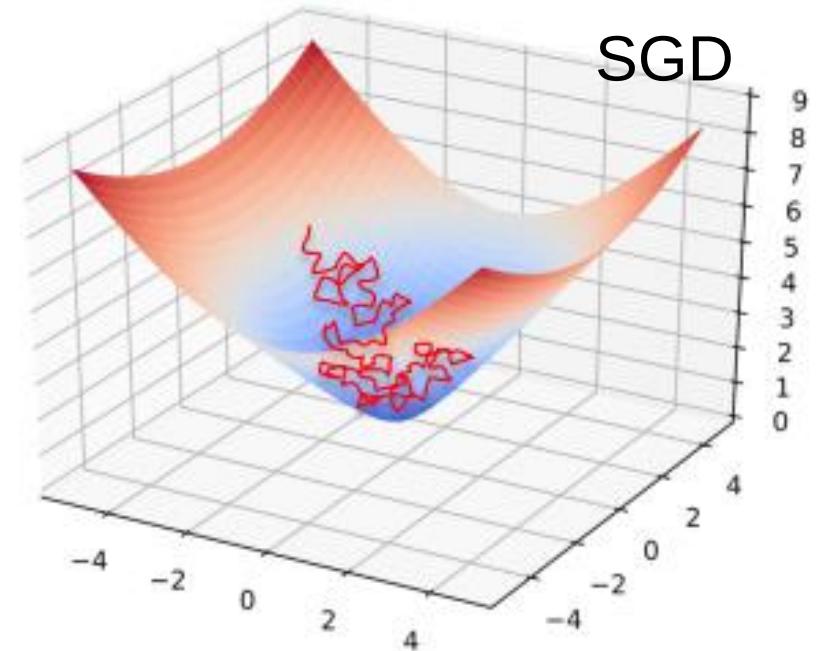
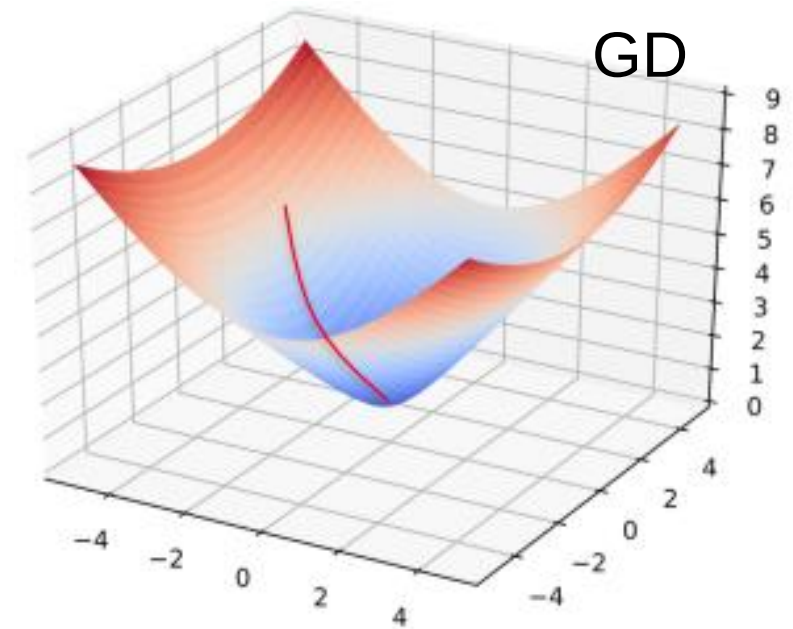
Stochastic Gradient Descent (SGD)

1. Initialize weights $W \leftarrow W^{(0)}$
2. While not converged do:
3. $i \sim \text{Uniform}(\{1, 2, \dots, N\})$ Sampling a data point
4. Compute gradient $\nabla_W J^{(i)}(W) = \frac{\partial J^{(i)}(W)}{\partial (W)}$ Calculate of 1 data point
5. Update weights
$$W \leftarrow W - \alpha \cdot \nabla_W J^{(i)}(W)$$
6. Return W

Original objective of GD $J(W) = \sum_i^N J^{(i)}(W)$

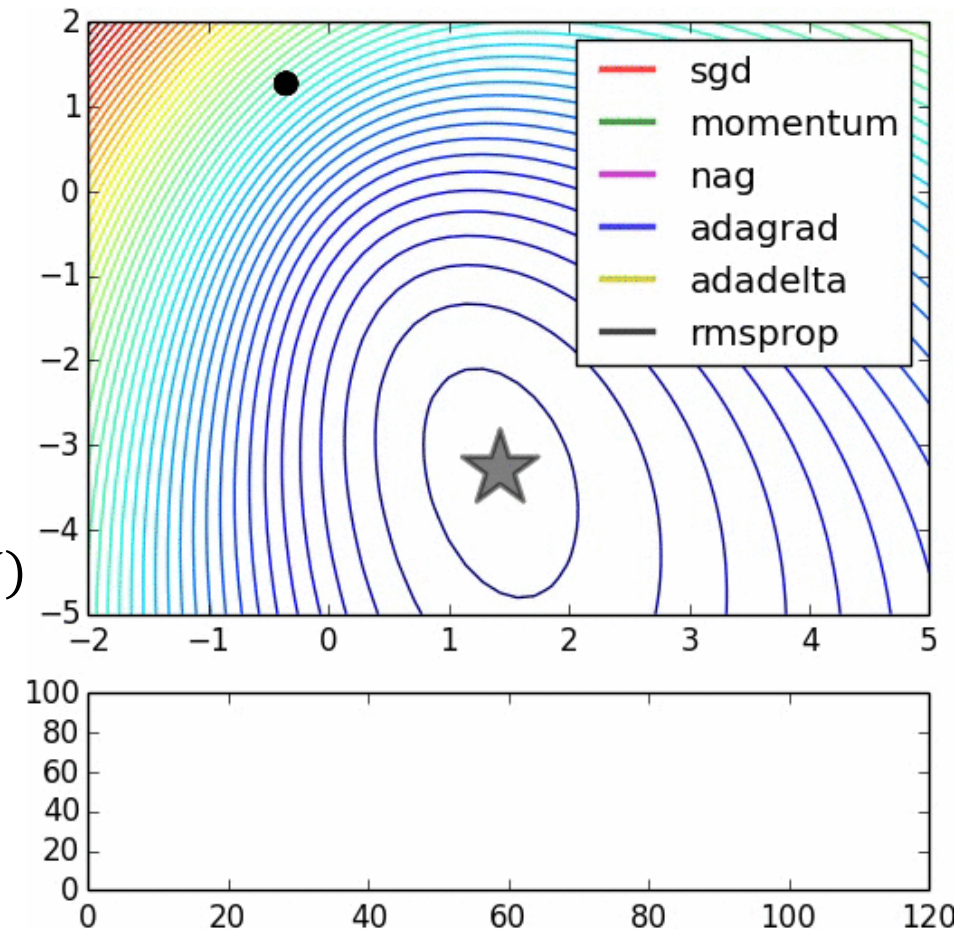
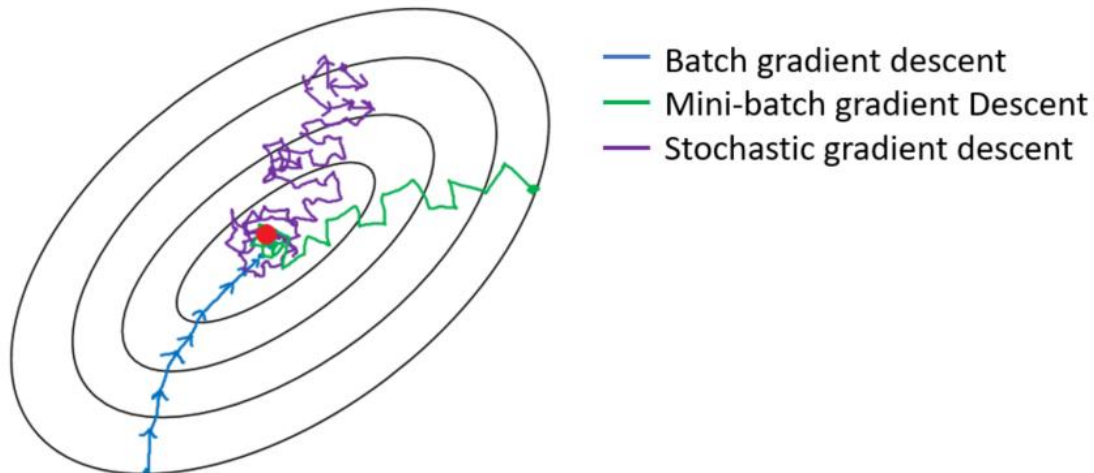
New objective of SGD $J^{(i)}(W)$

- Converge much faster!
- But performs frequent updates with a high variance that cause the objective function to **fluctuate** heavily



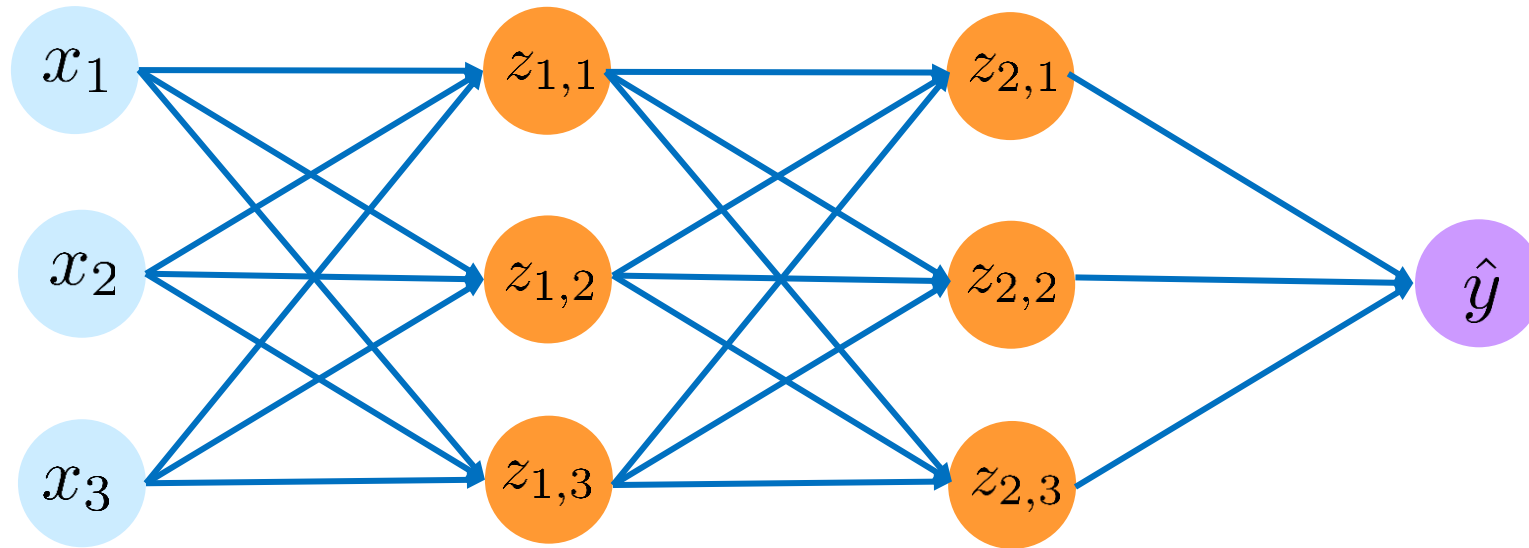
Mini-batch gradient descent

- Gradient descent
 - Stable descent on loss but converge slowly
- Stochastic gradient descent
 - Fast but have large variance of parameter update
- Mini-batch gradient descent
 - Sample a small batch of data to calculate loss
 - $$W \leftarrow W - \alpha \cdot \nabla_W J^{(i:i+n)}(W)$$
 - Take the best of both worlds – fast and stable!



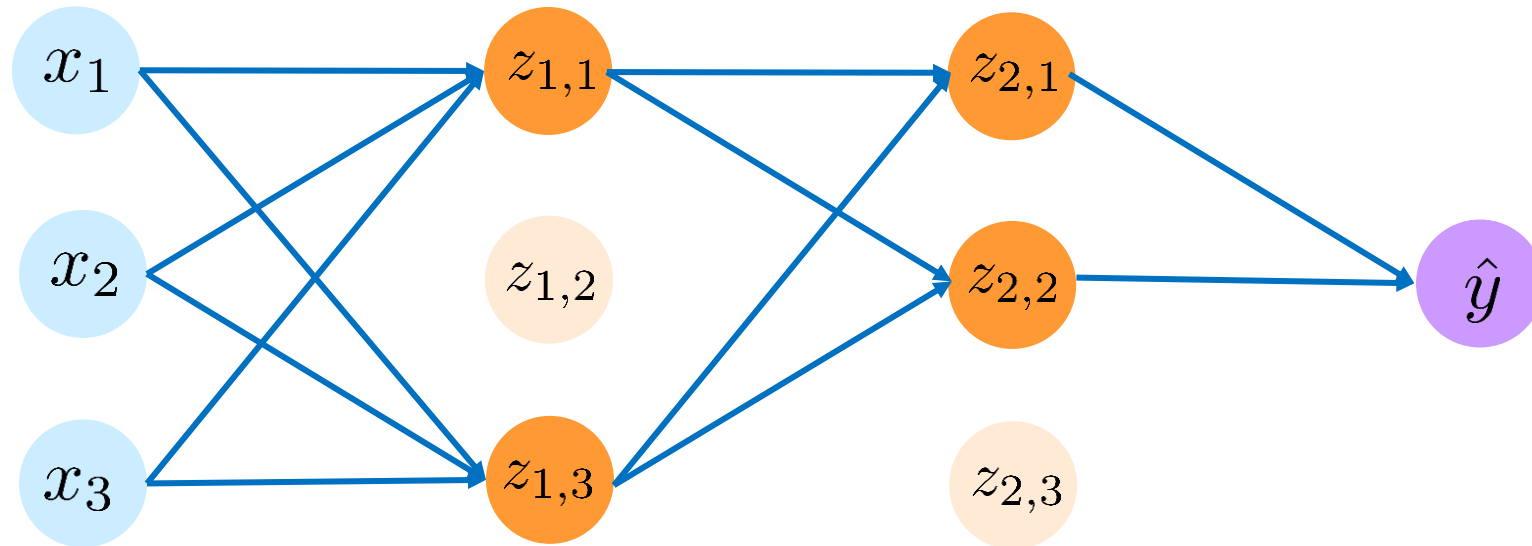
Model generalization

- **Goal:** Control overfitting, improve generalization of our model on unseen data
 - Dropout
 - Early stop
 - L2 Norm (make parameters close to 0)



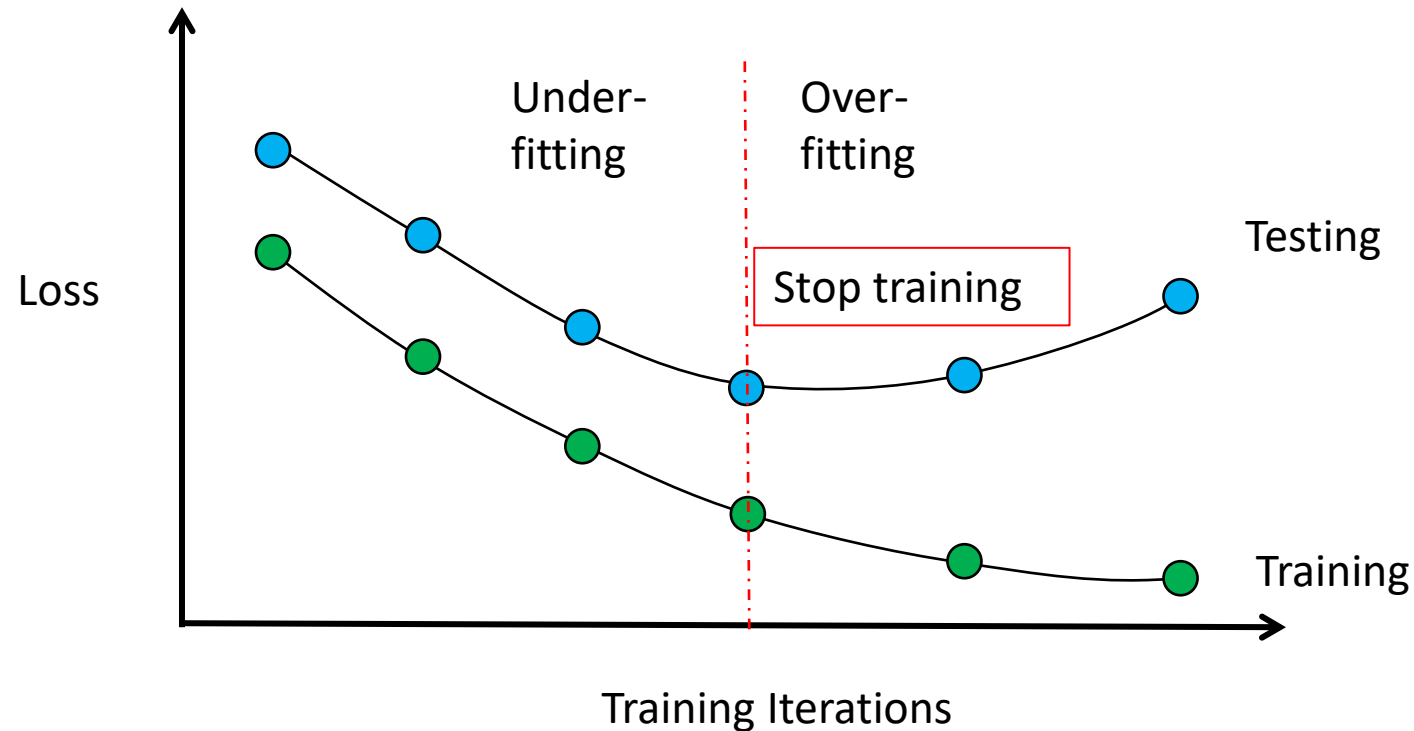
Dropout

- During training, randomly deactivate nodes to 0
 - For example, drop 30% of nodes in layer (they output 0)
 - Forces network to not rely on any single node (each node captures generalized information)
 - They don't perform during testing



Early stop

- Stop training before we have a chance to overfit
 - Monitor training/validation loss during training
 - Trigger if loss reduce reaches threshold



GPU vs. CPU



<https://youtu.be/-P28LKWTzrI?si=rLVD3Lyh9lcO6o49>